

RESEARCH ARTICLE

Automated Software Test Case Generation Using Directional Partially Weighted Ensemble Large Language Models With Retrieval-Augmented Generation (RAG)

WANTANA SISOMBOON^{ID}, JAKKRIT KAEWYOTHA^{ID}, AND WARARAT SONGPAN^{ID}

Department of Computer Science, College of Computing, Khon Kaen University, Khon Kaen 40002, Thailand

Corresponding author: Wararat Songpan (wararat@kku.ac.th)

This work was supported by the College of Computing, Khon Kaen University, Khon Kaen, Thailand.

ABSTRACT Ensuring software quality relies on testing practices that can reliably confirm whether delivered systems meet user requirements. While Acceptance Test-Driven Development (ATDD) encourages early collaboration between stakeholders and developers, creating comprehensive acceptance test cases remains a human task that heavily depends on domain expertise. Recent progress in Large Language Models (LLMs) has made automated test case generation feasible. However, approaches based on a single model often suffer from performance instability, uneven test coverage, and a limited ability to generalize across heterogeneous requirements. This study proposes an automated test case generation framework that combines directional partially weighted ensemble large language models (LLMs) with Retrieval-Augmented Generation (RAG). The framework assigns model influence dynamically according to directional performance trends and degrees of partial correctness and consolidates candidate outputs through a weighted majority-voting mechanism and cross-validation conducted over multiple folds. Five state-of-the-art LLMs, Claude, ChatGPT, Gemini, Qwen, and Mistral, were evaluated using 16 criteria derived from four widely adopted closed box testing techniques: Boundary Value Analysis, Equivalence Partitioning, Use Case testing, and State Transition testing. Under the evaluated experimental conditions, the proposed framework achieved 98.44% test coverage, indicating complete consistency with the defined gold-standard acceptance criteria. Cross-validation results further demonstrate improved robustness, stronger generalization to previously unseen requirements, and linear scalability, supporting practical deployment in real-world software development environments, and complexity analysis to confirm linear throughout growth ($O(n)$), which supports efficient deployment under increasing workloads.

INDEX TERMS LLM, RAG, weighted ensemble, directional partially, test coverage, ATDD, automated software testing.

I. INTRODUCTION

The software product market is increasingly competitive, with shortened development cycles and frequent product releases. While faster launches can improve user acquisition and satisfaction, quality validation is often postponed to later

development stages, increasing the risk of software failures and unmet user requirements. To mitigate these risks, shift-left testing methodologies have been proposed, among which Test-Driven Development (TDD) is widely adopted. Acceptance Test-Driven Development (ATDD) further extends this paradigm by promoting continuous collaboration among business stakeholders, developers, and testers throughout the development lifecycle. By generating acceptance test cases

The associate editor coordinating the review of this manuscript and approving it for publication was Joao Bernardo Ferreira Sequeiros^{ID}.

early, alongside requirements elicitation, ATDD helps reduce downstream defects and limits late-stage quality issues [1].

Recent advances in Large Language Models (LLMs) have enabled partial automation of software testing tasks, including test case generation and fault correction. A systematic review [2] highlights the growing adoption of LLMs in these contexts. Despite this progress, significant limitations remain, particularly in integration and acceptance testing, where outputs must be evaluated against user expectations and system behavior. In practice, single-model LLM approaches often exhibit unstable performance, uneven test coverage, and limited generalization across heterogeneous requirements, necessitating continued human verification and validation.

Prior studies have also investigated the generation of test cases directly from software requirements using LLMs. These approaches substantially reduce User Acceptance Testing (UAT) effort and execution time, reporting speedups compared with manual testing while improving alignment with user requirements [3]. However, challenges persist, including inconsistent performance in use case-based scenarios, limited handling of state transition conditions, and insufficient coverage across complex requirement structures. Moreover, standard LLMs typically lack access to restricted or domain-specific information in real-world projects, which can result in incomplete or inaccurate test cases.

To address these limitations, this study investigates automated acceptance test case generation using Retrieval-Augmented Generation (RAG). By integrating LLMs with internal and external knowledge sources, such as project-specific documentation and restricted datasets, RAG enables context-aware generation while mitigating confidentiality and domain-specificity constraints. This process is supported by structured prompt templates that combine the PARTS framework with user story representations to improve contextual relevance and output consistency [4], [5].

Beyond software testing, RAG has been applied across multiple domains, including healthcare [6], [7], business intelligence [8], education [9], and agriculture [10], [11]. Within software engineering, recent work demonstrates RAG's potential to support test generation, defect detection and debugging across the Software Development Lifecycle (SDLC) [12], [13], [14], [15], [16], [17]. Nevertheless, previous studies have identified persistent retrieval and integration challenges that limit the reliability of RAG-based systems in complex workflows [18]. Furthermore, existing research has largely focused on isolated testing tasks rather than comprehensive acceptance test generation.

In addition, ensemble methodologies for LLMs have emerged as an effective means of improving robustness and output consistency. These approaches aggregate multiple models either at the architectural level through weight integration or mixture-of-experts, or at the output level via voting and routing strategies [19], [20]. Motivated by these advances, this study integrates RAG with a weighted

ensemble strategy to mitigate single-model instability, address partial correctness in the generated test cases, and improve coverage consistency and generalization.

To overcome the limitations of single-model approaches in automated acceptance testing, this study addresses persistent challenges related to model instability, coverage inconsistency, and generalization in acceptance testing. By adopting an ensemble-based perspective and integrating Retrieval-Augmented Generation, the proposed approach improves the reliability and practical applicability of automated acceptance test case generation.

The main contributions of this study are as follows:

- 1) **Automated Acceptance Test Generation Framework:** This study presents an automated framework that integrates Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) to generate acceptance test cases from software requirements expressed in business and technical terminology.
- 2) **Directional Partially Weighted Ensemble Strategy:** A performance-aware ensemble mechanism is introduced to aggregate outputs from multiple LLMs based on directional performance behavior and degrees of partial correctness, mitigating single-model instability and improving coverage consistency.
- 3) **Comprehensive Empirical Evaluation:** An extensive evaluation is conducted using five state-of-the-art LLMs and 16 criteria across four established closed box testing techniques: Boundary Value Analysis (BVA), Equivalence Partitioning (EP), Use Case (UC), and State Transition (ST). The results indicate that weighted three-model ensembles improve coverage stability, while four-model ensembles exhibit stronger generalization to unseen requirements.
- 4) **Benchmark Metrics for Reproducibility:** Baseline metrics, including accuracy, precision, recall, F1-score, and test coverage, are reported to support reproducibility and serve as reference benchmarks for future research on LLM-based automated test case generation.

The remainder of this paper is organized as follows. Section II presents a background study of the related work. Section III provides reviews of related work on ensemble learning, Retrieval-Augmented Generation, and LLM-based software testing. Section IV describes the proposed methodology and experimental framework. Section V reports the experimental results, and Section VI provides a discussion of the findings. Finally, Section VII concludes the paper and outlines directions for future research.

II. BACKGROUND STUDY

A. SOFTWARE TESTING OVERVIEW

1) ACCEPTANCE TEST-DRIVEN DEVELOPMENT (ATDD) AND SHIFT-LEFT TESTING

Acceptance Test-Driven Development (ATDD) evolved from Test-Driven Development (TDD), introduced by Kent Beck

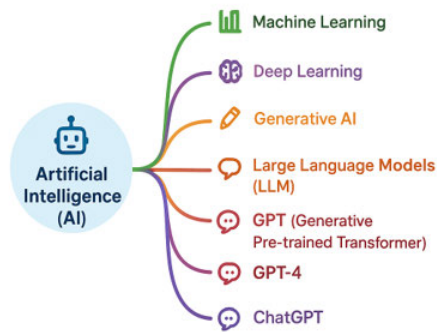


FIGURE 1. LLM Generative in the Context of AI Overview.*Source: Author (with assistance from ChatGPT, OpenAI).

in 2002, which emphasizes writing tests before implementing code. It is a collaborative methodology in which stakeholders, developers, and testers work together throughout the software development process to ensure alignment with customer requirements. Testing activities are moved to the early stages of development, alongside analysis and design, before coding begins; this approach is commonly referred to as shift-left. Shift-left testing reduces production issues, improves collaboration, and supports automation to enhance overall software quality [21], [22], [23].

2) USER ACCEPTANCE TESTING (UAT) AND SOFTWARE TESTING LEVELS

Software testing is organized into hierarchical levels based on scope and objectives. Unit testing validates individual components in isolation and is typically performed by developers. Integration testing verifies interactions among components, databases, APIs, and external services using strategies such as bottom-up, top-down, or big-bang integration. System testing evaluates complete system behavior, including both functional tasks and non-functional characteristics.

Acceptance testing represents the final validation stage before deployment and determines whether the system satisfies business requirements and user needs. It includes user acceptance testing (UAT), as well as operational, contractual, regulatory, alpha, and beta testing. Each testing level differs in objectives, test environment, defect focus, and responsibilities [24], [25].

This study focuses specifically on User Acceptance Testing (UAT), as UAT directly evaluates requirement conformance from the perspective of end users and stakeholders. Improving automation in UAT therefore has significant impact on overall software quality assurance.

B. LLM GENERATIVE AI

Large Language Models (LLMs) are deep neural networks designed to understand and generate natural language, as illustrated in Fig. 1. They are trained on large-scale corpora using self-supervised learning objectives, typically next-token prediction. Through this training paradigm, LLMs learn

contextual representations that capture syntactic, semantic, and pragmatic relationships within text [26], [27].

Generative AI systems, including LLMs, produce novel outputs such as text, code, or structured artifacts. Instead of retrieving predefined responses, LLMs generate outputs probabilistically based on learned token distributions. Recent generations support applications ranging from conversational agents and summarization tools to automated programming assistance and test-case generation [4], [26], [28].

Despite their versatility, standalone LLMs rely primarily on parametric memory encoded during training, which may lead to outdated knowledge or hallucinated content when domain-specific or real-time information is required. These limitations motivate the integration of external knowledge retrieval mechanisms.

C. RETRIEVAL-AUGMENTED GENERATION (RAG)

Retrieval-Augmented Generation (RAG) integrates a retriever module with a generative model. The retriever identifies relevant documents or knowledge fragments from external sources, such as document repositories or databases. The generator then conditions its output on both the original query and the retrieved evidence. By combining parametric knowledge (model weights) with non-parametric memory (external documents), RAG improves factual grounding and domain specificity [29], [30], [31].

Advances in dense retrieval methods and neural question-answering systems enabled tighter coupling between retrieval and generation components [32]. Empirical studies demonstrate that retrieval integration reduces hallucination and improves contextual alignment in knowledge-intensive tasks [4], [33]. Subsequent research introduced architectural optimizations, pre-training integration strategies, and benchmark standardization to enhance efficiency and adaptability [34], [35], [36], [37].

More recent work explores adaptive retrieval policies, reflection mechanisms, and structured reasoning pipelines to improve robustness and interpretability [29], [38]. Although RAG significantly enhances contextual accuracy, its effectiveness depends heavily on retrieval precision, document quality, and integration strategies—factors particularly relevant in structured domains such as software engineering.

III. RELATED WORKS

A. RAG IN SOFTWARE ENGINEERING

RAG has been increasingly adopted in software engineering for tasks including test generation, bug detection, vulnerability analysis, and documentation retrieval. Prior studies indicate that integrating retrieval mechanisms reduces hallucinations and improves factual consistency compared to standalone prompting approaches [39].

For automated test generation, domain-specific RAG frameworks demonstrate measurable improvements in contextual alignment between requirements and generated test cases [40]. However, performance often depends on

TABLE 1. Comparative studies on RAG applications in software testing.

Year & Authors	Application in Software Testing	RAG Methods	Key Insights
2024 - Arora, Herda, Homm [40]	Automated test scenario creation from NL requirements	RAG with GPT-3.5/4; zero/few-shot; glossary support	Few-shot improves precision; retrieval quality strongly affects output consistency.
2024 - Shin et al. [13]	Unit test generation for ML frameworks	API-level RAG with documentation and issue retrieval	Domain-specific retrieval increases coverage and reduces irrelevant outputs.
2025 - Wang et al. [42]	Bug detection and test generation during coding	Graph-based contextual RAG; IDE integration	Context-aware retrieval improves adaptability in evolving codebases.
2025 - Zhang et al. [46]	Vulnerability detection	Prompting; fine-tuning; ensemble strategies	Fine-tuning and ensembles outperform single prompting in complex cases.
2025 - Budakoğlu & Emekci [47]	Domain-specific vulnerability detection	Hybrid RAG with fine-tuning	Balanced trade-off between efficiency and precision.

input quality, glossary support, and retrieval configuration. API-level RAG approaches further enhance coverage by incorporating technical documentation and issue repositories, highlighting the importance of domain-relevant corpora [13]. Advanced frameworks such as QuIM-RAG further improve contextual consistency by aligning intermediate document-level questions with user queries, outperforming baseline RAG approaches [41]. Additional work explores structured embeddings and contextual graphs to support multi-file reasoning and adaptive bug detection workflows [42].

In vulnerability detection, hybrid approaches combining RAG with fine-tuning or ensemble strategies frequently outperform generic prompting, though they introduce higher computational and integration complexity [43], [44]. Comprehensive surveys emphasize reproducibility challenges, scalability concerns, and the need for standardized evaluation metrics in industrial contexts [45], [46], [47], [48].

Table 1 highlights methodological variations and empirical findings across representative RAG-based studies in software testing. While these works demonstrate performance gains, most focus on isolated components such as retrieval tuning, prompt design, or fine-tuning strategies rather than unified end-to-end ensemble coordination.

B. RAG ENSEMBLE AND MULTI-MODEL STRATEGIES

Ensemble techniques enhance robustness and generalization in LLM-based systems. Architectural-level ensembles include weight merging, knowledge fusion, and Mixture of Experts (MoE), whereas model-level ensembles aggregate outputs through voting, routing, or cascading mechanisms [20].

Recent research extends ensemble concepts to RAG systems. Multi-RAG architectures reduce uncertainty by combining outputs from diverse retrieval pipelines [49]. Other work investigates ensemble strategies for long-context

reasoning, improving stability in complex query scenarios [50]. Hybrid retrieval approaches that combine dense vector search and sparse keyword-based retrieval further enhance recall and contextual relevance [51], [52].

In software testing contexts, multi-LLM ensembles demonstrate improved coverage consistency and evaluation metrics compared to single-model configurations [53]. Nevertheless, empirical evaluations often lack standardized benchmarks and cross-domain validation [54]. Integration complexity, computational cost, and deployment trade-offs remain underexplored, particularly for acceptance testing workflows.

Despite these advances, key imitations persist. Empirical evaluations often lack cross-language benchmarking and standardized assessment protocols, reducing comparability and reproducibility. Integration complexity and computational trade-offs remain underexplored, particularly in end-to-end software testing environments. These gaps highlight the need for unified ensemble RAG frameworks that coordinate retrieval-level and model-level aggregation to improve robustness, contextual accuracy, and deployability in software engineering applications.

IV. RESEARCH METHODOLOGY AND PROPOSED FRAMEWORK

A. LLM-DRIVEN AUTOMATED SOFTWARE TEST CASE GENERATION USING WEIGHTED-VOTE ENSEMBLE MODELS WITH A RETRIEVAL-AUGMENTED GENERATION (RAG) ARCHITECTURE

This section outlines a methodology for employing multiple large language models (LLMs) in combination with a Retrieval-Augmented Generation (RAG) workflow for software test case generation by extracting relevant context from a vector database. Each LLM independently generates outputs with the RAG mechanism. Following RAG-based generation, ensemble aggregation is performed using a combination of majority voting and directional partially weighted ensemble techniques to improve accuracy and expand test coverage, as illustrated in Fig. 2. The process comprises the following steps:

- 1) DATA PREPARATION
 - Step 1: System Setup and Data Ingestion

a: DATASET CONSTRUCTION

The industrial datasets used in this study were derived from enterprise-level information systems across finance, logistics, and education domains. The requirements predominantly involved structured business-process workflows, state-transition logic, multi-step approval chains, and transactional validation rules. These characteristics reflect common acceptance testing scenarios in real-world enterprise software systems, where correctness depends on status transitions, event-driven updates, and role-based authorization logic.

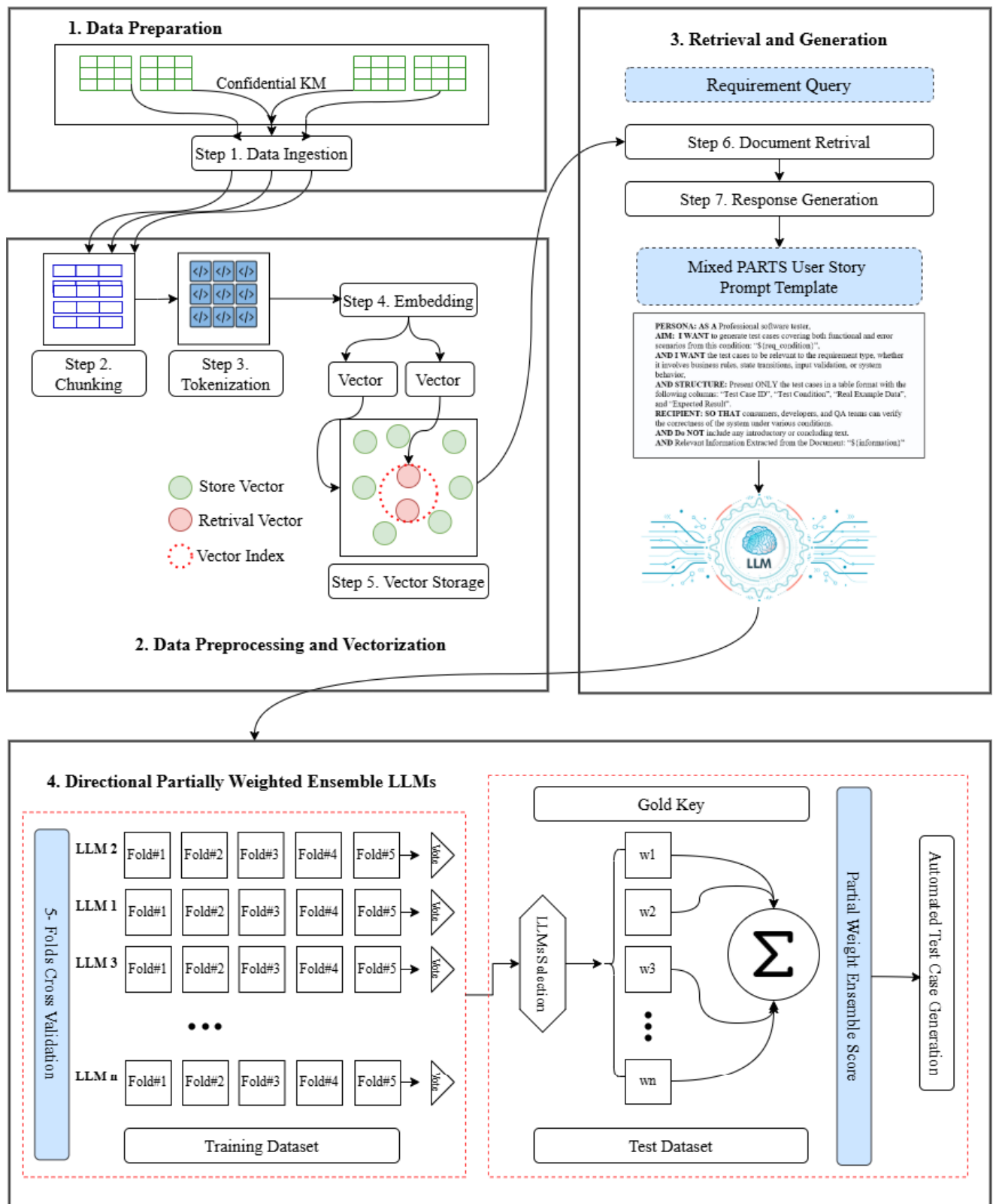


FIGURE 2. Architecture of the automated software test case generation framework using directional partially weighted ensemble large language models with retrieval-augmented generation.

No personally identifiable or proprietary business information is included in the released materials.

The dataset consists of 16 software requirements, each involving test case generation using four standard

software testing techniques: Boundary Value Analysis (BVA), Equivalence Partitioning (EP), Use Case, and State Transition. The requirements were sourced from anonymized industrial projects, Kaggle datasets, and materials from the International Software Testing Qualifications Board (ISTQB) (Table 2).

The training dataset was derived from a software testing course attended by over 1,000 professionals in Thailand during the past ten years. The requirements and corresponding test cases were refined based on global project experience and a decade of instructional practice. Due to limited access to real industrial data, only three out of ten partner software companies provided datasets, and none fully met the study objectives. Therefore, additional requirements were collected from public sources and paired with newly developed test cases. This process resulted in a novel dataset covering four standard software testing techniques.

The test dataset consisted of previously unseen requirements from diverse sources and was validated by three experienced software testers. It was constructed by combining open, reproducible datasets with standard testing materials to ensure both practical relevance and reproducibility.

During preprocessing, formatting inconsistencies were corrected while preserving natural variation in real-world requirements. This approach maintained data authenticity and improved model robustness. As part of retrieval preparation, balanced knowledge sources were carefully selected to mitigate bias in the Retrieval-Augmented Generation (RAG) process [39].

b: TRAINING AND TEST SPLIT

For experimental design, Requirements 1–8 were used for training, and Requirements 9–16 were used for testing. The number of test cases per requirement, ranging from 2 to 10, was determined according to complexity levels defined by a software testing expert with over 20 years of professional experience. Three manual testers with two to five years of experience developed and evaluated the test cases based on time spent and test coverage. The dataset included eight Level-1 (Very Low), six Level-2 (Low), one Level-3 (Moderate), and one Level-4 (High) requirements.

In this experiment, four spreadsheets served as raw data and were imported into a vector database to support efficient semantic retrieval. After ingestion, the data were organized according to software testing methodologies: (1) Equivalence Partitioning (EP) with 16 requirements and 77 test cases; (2) Boundary Value Analysis (BVA) with 12 requirements and 69 test cases; (3) Use Case with 12 requirements and 86 test cases; and (4) State Transition with seven requirements and 58 test cases.

c: IMPLEMENTATION ENVIRONMENT

The RAG system was implemented on a Windows 11 (64-bit) laptop with an Intel 1.30 GHz processor and 16 GB RAM. The framework utilized FAISS and ChromaDB for vector retrieval, Sentence-Transformers for embedding generation, LangChain for orchestration, and OpenAI API (via OpenRouter) for LLM inference. Supporting libraries included Pandas, PyPDF, Python-Docx, PyTesseract, and Matplotlib for data processing and analysis.

2) DATA PREPROCESSING AND VECTORIZATION

- **Step 2: Text Chunking** Long texts are segmented into smaller units based on tokens, sentences, or paragraphs. In practice, character-level splitters such as LangChain's `RecursiveCharacterTextSplitter` and `FixedWindowTextSplitter`, as well as tokenizers such as `pythainlp.word_tokenize` for Thai, partition content while preserving semantic integrity. The chunking function utilizes two parameters: chunk size and chunk overlap. A chunk size of 1,000 characters and an overlap of 200 characters were applied in this experiment.

The chunking function is parameterized by chunk size and chunk overlap. In this experiment, `chunk_size` was set to 1,000 characters, and `chunk_overlap` was set to 200 characters. Each resulting chunk is stored in an intermediate format, such as JSON, to facilitate token-based splitting.

- **Step 3: Text Chunking**

Raw requirement documents were segmented into smaller semantic units to optimize retrieval granularity. A chunk size of 1,000 characters with 200-character overlap was applied using LangChain's `RecursiveCharacterTextSplitter`. Tokenization for Thai content was performed using `PyThaiNLP`. Each chunk was stored as an independent retrieval unit.

- **Step 4: Embedding Creation**

Token-based segments were converted into high-dimensional numerical vectors using pre-trained embedding models, including OpenAI's text-embedding-3-small and Hugging Face's all-MiniLM-L6-v2.

The OpenRouter API was utilized to enable flexible access to embedding providers through secure environment-based authentication.

Each embedding vector supports efficient similarity computation during retrieval to identify contextually relevant content.

$$\mathbf{v}_t = \text{embed}(\text{requirement}_t) \in \mathbb{R}^d \quad (1)$$

where $f_{\text{embed}}(\cdot)$ denotes the embedding function, d represents the embedding dimension, and $\mathbf{v}_t$ is the dense vector representation of requirement t .

Raw text and corresponding embeddings were stored together to ensure traceability.

- **Step 5: Vector Database Storage** Embeddings are stored in a vector database, such as ChromaDB or FAISS. These databases organize data based on semantic meaning rather than lexical content, thereby enhancing information retrieval. This configuration enables hybrid search capabilities, combining keyword filtering with semantic search. The system retrieves results that align with both the query's textual content and its intent. Each embedding represents the semantic meaning of its corresponding text, and the database clusters similar items using cosine similarity to facilitate rapid search. This approach ensures that user queries yield contextually relevant results, independent of specific wording. Each database collection groups similar documents, and each entry contains a unique identifier, its embedding, and associated metadata (source, index, model), thereby maintaining data traceability. Standardized naming conventions were applied to collections to ensure consistency. The `OPENROUTER_API_KEY` environment variable, specified in the `.env` configuration file, was utilized to dynamically manage model access. This setup supports multiple model providers and maintains secure, model-independent access. The system serves as a reusable knowledge base for RAG workflows, enabling efficient, context-aware retrieval across multiple models and datasets.

3) RETRIEVAL AND GENERATION

- **Step 6: Document Retrieval**

The retrieval module indexes structured testing knowledge, including historical test cases, requirement specifications, and technique-specific templates (EP, BVA, Use Case, State Transition), using embedding-based vector representations. User requirements are converted into embedding vectors using the same model applied during data preparation.

The generated vectors are compared with those stored in the database using similarity metrics such as Euclidean distance or cosine similarity. In this study, cosine similarity was adopted due to its effectiveness. The similarity threshold τ was optimized via grid search within the range of 0.60 to 0.90 using a 20% validation set. The optimal threshold, $\tau = 0.75$, achieved the highest F1-score and was applied in all subsequent experiments.

For each requirement, the system retrieves the top-5 most semantically aligned chunks based on cosine similarity ranking, balancing precision and computational efficiency.

The retrieved artifacts are appended to the generation prompt to provide contextual grounding. Retrieval effectiveness was verified during calibration to ensure semantic coherence and prevent irrelevant augmentation, thereby supporting the LLM in generating precise test cases.

TABLE 2. Experimental dataset for test case analysis.

Req.	Technique	Level	Data Source	Business Domain	Total TC
Training Dataset					
1	BVA	1	Real Project	Banking and Financial Services	3
2	EP	1	Real Project	Banking and Financial Services	2
3	BVA	2	Real Project	Banking and Financial Services	5
4	BVA (2 conditions)	2	Real Project	Banking and Financial Services	8
5	Use Case	3	Real Project	Banking and Retail Integrations	2
6	BVA	1	Real Project	Banking	3
7	BVA	1	Real Project	Banking	3
8	State Transition	4	Real Project	E-Commerce and Retail	4
Test Dataset					
9	EP	1	Kaggle [56]	Automotive	6
10	EP	1	Kaggle [56]	Logistics	7
11	EP	1	ISTQB [24], [25]	Generic Business Application	3
12	BVA	1	ISTQB [24], [25]	E-Commerce and Retail	5
13	Use Case	2	Kaggle [56]	Education Management System	4
14	Use Case	2	Kaggle [56]	Gaming and Entertainment Software	4
15	State Transition	3	Real Project	Industrial Workflow System	7
16	State Transition	3	Real Project	E-Commerce and Retail	10

Note: Real project data were provided by the Siam Chamnankit Company.

The training datasets used in this study are publicly available on GitHub at: <https://github.com/wararatkku/rag-ensemble-acceptance-test-generation>

- **Step 7: Response Generation** In this step, the tester selects an appropriate software testing technique and provides the required inputs. The framework then generates the results using a structured, step-by-step prompt template. Previous studies have shown that generative AI models can automatically generate user acceptance test (UAT) cases through systematic prompt engineering approaches. These approaches integrate the PARTS technique with the Agile User Story format within a Zero-Shot Prompting framework [40], [55]. Mixed prompt templates are illustrated in Fig. 3 and were designed by combining key components, including task, context, role, format, tone, examples, and constraints. These components were aligned with Google's PARTS framework, which consists of Persona, Aim, Recipients, Theme, and Structure. Furthermore, following the Agile Alliance and Mike Cohn's User Stories Applied for Agile Software Development, the User Story format is defined as: "As a [role], I want [goal], so that [purpose]." Additional conditions can be incorporated using "AND" to enhance expressiveness while maintaining consistency with Agile and Extreme Programming (XP) practices. These Mixed prompt templates were derived from and synthesized based on the original prompt patterns presented in Fig. 8 in Appendix. To further enhance response quality, this study incorporates Retrieval-Augmented Generation (RAG), which

TABLE 3. Selected LLM models.

No.	Model Name	Provider	Params	Max Tokens
1	Claude-Sonnet-4	Anthropic	300–500B (608B)	Up to 64K output
2	ChatGPT-4o	OpenAI	1.8T	128K (input + output)
3	Gemini-2.5-Pro	Google	288B	In: 1,048,576; Out: 65,535
4	Qwen3-Coder	Qwen	480B	256K (to 1M)
5	Mistral-Small-3.2-24B-Instruct	MistralAI	24B	Up to 64K
6	GLM-4.5V	Zhipu AI	355B	Up to 131,072
7	Sonar-Pro	Perplexity	70B	Up to 200K
8	DeepSeek-R1-0528	DeepSeek	671B	Up to 163,840

PERSONA: AS A Professional software tester.
AIM: I WANT to generate test cases covering both functional and error scenarios from this condition: “\${req_condition}”
AND I WANT the test cases to be relevant to the requirement type, whether it involves business rules, state transitions, input validation, or system behavior.
AND STRUCTURE: Present ONLY the test cases in a table format with the following columns: “Test Case ID”, “Test Condition”, “Real Example Data”, and “Expected Result”.
RECIPIENT: SO THAT consumers, developers, and QA teams can verify the correctness of the system under various conditions.
AND Do NOT include any introductory or concluding text.
AND Relevant Information Extracted from the Document: “\${information}”

FIGURE 3. Mixed PARTS User Story template.

extracts a clear query from the user input and retrieves relevant information from an external knowledge base [56]. The retrieved knowledge is injected into the prompt, enabling the AI model to generate more accurate, consistent, and reliable responses. In addition, well-defined prompt structures and appropriate dataset selection help reduce bias and improve fairness in the generated outputs.

The completed prompt is sent to the selected LLM via an API request. The generated output is post-processed and presented in structured test-case tables, integrating retrieved knowledge with generative AI to produce comprehensive and reusable software testing documentation. In this experiment, the prompt response results were organized in a table consisting of test case ID, test condition, real example data, and expected result.

4) DIRECTIONAL PARTIALLY WEIGHTED ENSEMBLE LLMS

A weighted vote ensemble was proposed to improve the reliability of test case generation by reducing model-specific biases while combining the strengths of multiple LLMs.

Models were selected from the top 20 of 499 text-generative AI models on the OpenRouter leaderboard (Table 3) based on token usage metrics (prompt, completion, and reasoning tokens) and alignment with tools currently used by industry partners. Usage statistics were collected between June and August 2025.

For each software requirement, all selected models generated candidate outputs through a Retrieval-Augmented

Generation (RAG) pipeline, enhanced using context retrieved from a vector database. Model-specific retrieval behavior ensured alignment with the reasoning style of each model.

The generated outputs were evaluated against the gold standard using semantic similarity filtering. Only results exceeding a predefined similarity threshold were retained. The outputs were regenerated multiple times to ensure independence and consistency. Subsequently, a majority-vote ensemble was applied to identify the models with the highest coverage and accuracy.

Gold standards were constructed and validated for each dataset source. The training dataset gold standard was adapted from real-world projects and validated by a software tester trainer with over 20 years of professional experience. This gold standard has been used to train software testers for more than ten years across various business domains, including finance, banking, and education.

The gold standards for the ISTQB datasets were derived from official ISTQB qualification examinations and validated against the corresponding answer manuals. These examination materials are used globally to certify professional software testers.

The gold standards for the Kaggle datasets were constructed and evaluated by three software testing lecturers with 3–10 years of experience in higher education. The real-world company dataset was constructed by a professional software testing team within the organization and validated by the same academic experts, together with a senior software tester from Siam Chamnankit Company with over ten years of professional experience.

The final evaluation employed a directional partially weighted ensemble technique. The prediction of each model was converted into a numerical score: 1.0 for full correctness, 0.5 for partial correctness, and 0.0 for incorrect predictions. The model weights were derived from average performance metrics, including accuracy and F1-score, and normalized to sum to 1. Aggregated weighted scores were used to classify each test case into three categories: pass/related, partial pass/partially related, or fail/not related.

• Five Folds Model Selection Utilizing Majority-Vote Ensemble and Model-Specific RAG Matching

The evaluation procedure for the eight selected models, denoted as $R[1..8]$, is presented in Algorithm 1. The protocol consisted of five iterations incorporating RAG-based retrieval from dataset D , model-specific retrieval strategies, semantic similarity filtering using $TempThreshold[m]$, and a majority-vote ensemble to identify the top five models based on final coverage (C_{final}) and accuracy (A_{final}).

Each model processed all eight requirements independently across iterations. Generated outputs were enhanced using retrieved documents aligned with each model’s retrieval approach. Outputs with semantic similarity below 0.8 relative to the gold standard ($GoldKey$) were excluded.

Algorithm 1 K-Folds Model Selection Utilizing Majority-Vote Ensemble and Model-Specific RAG Matching

Require: $D, R[1..8]$ // Requirements 1 to 8, $M[1..8]$, Gold-Key, TempThreshold[m]

Ensure: Top 5 Models

```

1: Results  $\leftarrow \emptyset$ 
2: for each model  $m$  in  $M$  do
3:   AllOutputs  $\leftarrow \emptyset$ 
4:   for  $i = 1$  to 5 do
5:     CreateNewTask( $m$ )
6:     IterationOutputs  $\leftarrow \emptyset$ 
7:     for each requirement  $r$  in  $R$  do
8:       GeneratedOutputs  $\leftarrow$  ExecuteModel( $m, r$ )
9:       RAGOutputs  $\leftarrow$  RetrieveFromRAG( $r, D, ModelBehavior[m]$ )
10:      CandidateOutputs  $\leftarrow$  GeneratedOutputs  $\cup$  RAGOutputs
11:      MatchedOutputs  $\leftarrow \emptyset$ 
12:      for each  $cOutput$  in CandidateOutputs do
13:        similarity  $\leftarrow$  SemanticSimilarity( $cOutput, GoldKey[r]$ )
14:        if similarity  $\geq$  max(TempThreshold[ $m$ ], 0.8) then
15:          Append  $cOutput$  to MatchedOutputs
16:        else
17:          Flag  $cOutput$  as Invalid
18:        end if
19:      end for
20:      Append MatchedOutputs to IterationOutputs
21:    end for
22:    Append IterationOutputs to AllOutputs
23:  end for
24:  FinalOutputs  $\leftarrow$  MajorityVote(AllOutputs)
25:   $C_{final} \leftarrow$  ComputeCoverage(FinalOutputs, GoldKey)
26:   $A_{final} \leftarrow$  CalculateAccuracy(FinalOutputs, GoldKey)
27:  Append ( $m, C_{final}, A_{final}$ ) to Results
28: end for
29: Sort Results in descending order of  $C_{final}$ , then by  $A_{final}$ 
30: Top5Models  $\leftarrow$  first five entries of Results
31: Exclude all flagged Invalid cases from FinalOutputs
32: return Top5Models
  
```

Coverage and accuracy were computed, and models were ranked primarily by coverage and secondarily by accuracy. The algorithm exhibits linear scalability, with time and space complexity defined as:

$$\text{TimeComplexity} = \mathcal{O}(mknt) \quad (2)$$

$$\text{SpaceComplexity} = \mathcal{O}(mkn) \quad (3)$$

where m , k , n , and t denote the number of models, the number of folds (5), the number of requirements, and the average number of candidate outputs per requirement, respectively.

B. EVALUATION METRICS

Evaluating Retrieval-Augmented Generation (RAG) systems is challenging due to their hybrid nature and reliance on dynamic knowledge sources [49]. Recent investigations have introduced systematic frameworks, such as Auepora [57], which categorize evaluation into retrieval, generation, and system-level components. These frameworks emphasize targets including relevance, accuracy, faithfulness, correctness, latency, diversity, and reliability. Instruments like the TruEra RAG Triad, LangChain Bench, and Databricks Eval, alongside benchmarks such as RAGAs, ARES, RGB, and DomainRAG, achieve these goals through a combination of conventional metrics (e.g., MAP, MRR, BLEU, ROUGE, F1) and innovative LLM-as-judge approaches. The QA corpora, such as HotpotQA and FEVER, were used to construct domain-specific, real-time datasets, illustrating the shift towards thorough, user-focused evaluation. Recent contributions encompass RAG Playground [58], which demonstrates the effectiveness of hybrid retrieval methodologies and structured self-evaluation prompting, as well as LONG²RAG [59], which introduces long-context benchmarks and metrics such as Key Point Recall (KPR), Key Point Precision (KPP), and Key Point F1 (KPF) for a more nuanced evaluation. This study employed a weighted performance ensemble approach, combined with alternative assessment methods and techniques, to enhance reliability by integrating various LLM outputs into a cohesive framework for experimental analysis.

1) TEST COVERAGE CALCULATION

The model outputs were evaluated against the gold standard for each test case to calculate the Test Coverage (%), with each prediction classified as follows:

- True Positive (TP): The test case is intended to produce a correct result, and the model prediction fully matches the expected outcome.
- False Positive (FP): The test case is intended to produce a correct result, but the model predicts an incorrect or unrelated outcome.
- False Negative (FN): The test case is intended to produce a correct result, but the model fails to predict it or provides no answer.
- True Negative (TN): The test case is intended to produce an error or be irrelevant, and the model correctly predicts the error or irrelevance.
- TP Partial/TN Partial: This model produces a partially correct result, satisfying some but not all conditions of the requirement. Partial matches were assigned a score of 0.5, an accuracy of approximately 50%. These values were adjusted according to the evaluation criteria.
- Related: implies a functional test case, the model prediction response that was rejected as the gold key answer, but is relevant to the requirement, or non-functional test cases relevant to the requirement.

The Test Coverage was calculated as follows:

$$\text{Coverage} = \frac{\text{TP} + \text{TN} + 0.5(\text{TP}_{\text{Partial}} + \text{TN}_{\text{Partial}})}{N} \times 100 \quad (4)$$

where N denotes the total number of test cases.

Test Coverage (%) quantitatively assesses the model's ability to generate accurate test-case outputs, excluding partially correct responses. Test cases were generated for each LLM across five independent rounds, after which a majority-vote mechanism was applied to determine the final categorization for each test case. The resulting categories (TP, FP, FN, TN, Partial TP, and Partial TN) were then used to compute Test Coverage (%), enabling a comprehensive evaluation of model performance across multiple iterations.

This computation was subsequently applied to evaluate coverage results derived from the N-Model Weight framework, thereby ensuring a consistent assessment of ensemble or weighted-model outputs.

2) ACCURACY, RECALL, PRECISION, F1-SCORE

The evaluation metrics are defined as follows:

- **Accuracy** measures the proportion of correctly predicted instances out of all evaluated predictions, including partially correct cases.

$$\text{Accuracy} = \frac{\text{TP} + \text{TP}_{\text{Partial}} + \text{TN} + \text{TN}_{\text{Partial}}}{\text{TP} + \text{TP}_{\text{Partial}} + \text{TN} + \text{TN}_{\text{Partial}} + \text{FP} + \text{FN} + \text{Related}} \quad (5)$$

However, the difference between accuracy and coverage lies in their respective scopes: accuracy considers only the relevant cases, whereas coverage encompasses all cases, including both relevant and non-relevant instances.

- **Precision** measures the proportion of correct and relevant answers retrieved by the generative AI model among all retrieved answers.

$$\text{Precision} = \frac{\text{TP} + \text{TP}_{\text{Partial}} + \text{Related}}{\text{TP} + \text{TP}_{\text{Partial}} + \text{FP} + \text{Related} + \text{NotRelated}} \quad (6)$$

For example, when conducting a ChatGPT4 search for “\$req1”, if the keyword appeared in 5 of the 10 test cases displayed, the precision was 5/10. Precision is also useful for addressing bias and improving the system's stability in the evaluation framework.

- **Recall** measures the proportion of relevant answers retrieved by Generative AI compared to the total number of correct answers available (Ground Truth).

$$\text{Recall} = \frac{\text{TP} + \text{TP}_{\text{Partial}}}{\text{TP} + \text{TP}_{\text{Partial}} + \text{FN} + \text{Related}} \quad (7)$$

- **F1-score** is the harmonic mean of Precision and Recall:

$$F_1 = 2 \frac{\text{Precision} \text{ Recall}}{\text{Precision} + \text{Recall}} \quad (8)$$

3) WEIGHTED ENSEMBLE SCORING

After evaluating model performance using a majority-vote ensemble technique, a directional partially weighted ensemble approach was applied to refine the final decision for each test instance. This method proportionally represents the performance contributions of individual models, thereby improving robustness and reducing the influence of weaker models.

• Step 1: Model Selection

Let the set of selected models be denoted by S , where $|S| \in \{3, 4, 5\}$. Models with the highest average test coverage were selected and ranked in descending order.

• Step 2: Test Dataset Evaluation

For each model $i \in S$, a performance score $\bar{m}_i$ was computed using the training dataset. This score was defined as the average of Accuracy _{i} and F1 _{i} :

$$\bar{m}_i = \frac{\text{Accuracy}_i + F1_i}{2} \quad (9)$$

The corresponding model weights were then calculated using a numerically stable softmax function with temperature parameter $\lambda > 0$ (set to $\lambda = 1$ in this study):

$$w_i = \frac{\exp\left(\frac{\bar{m}_i - \max_{k \in S} \bar{m}_k}{\lambda}\right)}{\sum_{j \in S} \exp\left(\frac{\bar{m}_j - \max_{k \in S} \bar{m}_k}{\lambda}\right)} \quad (10)$$

• Step 3: Prediction-to-Score Conversion

Each model's prediction for a given test case was converted into a numerical score to enable weighted aggregation and comparative evaluation. The scoring scheme was defined as follows:

$$\begin{aligned} \text{Score}_i[t] &= \begin{cases} 1.0, & \text{if the outcome is TP or TN} \\ 0.5, & \text{if the outcome is TP}_{\text{Partial}} \text{ or TN}_{\text{Partial}} \\ 0.0, & \text{if the outcome is FP or FN} \end{cases} \end{aligned} \quad (11)$$

This scheme assigns the highest score to fully correct predictions, a partial score to partially correct predictions, and zero to incorrect predictions.

The fixed partial correctness weight was determined through preliminary empirical calibration and expert validation to reflect semantically meaningful but incomplete test cases. This intermediate value ensures proportional contribution during ensemble aggregation without equating partial alignment with full correctness. Sensitivity analysis indicated that moderate variations in the partial weight did not significantly affect ensemble ranking trends, suggesting that the framework is not overly dependent on a single finely tuned parameter.

• Step 4: Final Ensemble Score

The final directional partially weighted ensemble score for each test case t was computed as:

$$\text{FinalScore}[t] = \sum_{i \in S} w_i \text{Score}_i[t] \quad (12)$$

Algorithm 2 Directional Partially Weighted Ensemble Evaluation

Require: Selected models S , where $|S| \in \{3, 4, 5\}$;
 Software requirement r ; generated test cases t ;
 Semantic thresholds $\theta_{\text{sem}}, \theta_{\text{rel}}$

Ensure: Final label for each test case t

```

1: for each model  $i \in S$  do
2:   Obtain  $\text{Accuracy}_i$  and  $\text{F1}_i$  from training data
3:    $\bar{m}_i \leftarrow \frac{\text{Accuracy}_i + \text{F1}_i}{2}$ 
4: end for
5:  $m_{\text{max}} \leftarrow \max_{k \in S} \bar{m}_k$ 
6: for each model  $i \in S$  do
7:   Compute normalized softmax weight:
8:    $w_i \leftarrow \frac{\exp(\bar{m}_i - m_{\text{max}})}{\sum_{j \in S} \exp(\bar{m}_j - m_{\text{max}})}$ 
9: end for
10: for each test case  $t$  do
11:   for each model  $i \in S$  do
12:     Determine prediction outcome
13:     Convert outcome to numerical score:
14:      $\text{Score}_i[t] \in \{1.0, 0.5, 0.0\}$ 
15:     Compute similarity  $\text{sim}_i(r, t)$ 
16:   end for
17:   Compute ensemble score:
18:    $\text{FinalScore}[t] \leftarrow \sum_{i \in S} w_i \text{Score}_i[t]$ 
19:   Compute semantic similarity:
20:    $\text{sim}(r, t) \leftarrow \sum_{i \in S} w_i \text{sim}_i(r, t)$ 
21:   Evaluate functional correctness  $f(t) \in \{0, 1\}$ 
22:   Evaluate completeness  $c(t) \in \{0, 1\}$ 
23:   if  $\text{sim}(r, t) \geq \theta_{\text{sem}} \wedge f(t) = 1 \wedge c(t) = 1$  then
24:     decision  $\leftarrow$  Pass
25:   else if  $\text{sim}(r, t) \geq \theta_{\text{sem}} \wedge f(t) = 1 \wedge c(t) = 0$  then
26:     decision  $\leftarrow$  Partial Pass
27:   else if  $\theta_{\text{rel}} \leq \text{sim}(r, t) < \theta_{\text{sem}} \wedge f(t) = 0$  then
28:     decision  $\leftarrow$  Related
29:   else
30:     decision  $\leftarrow$  Not Related
31:   end if
32: end for
33: return Final decisions

```

This aggregation emphasizes models with higher reliability while still incorporating the contributions of all selected models. The resulting $\text{FinalScore}[t]$ lies within the interval $[0, 1]$, where higher values indicate stronger model agreement and confidence.

- **Step 5: Decision-rule-based Evaluation Mechanism**

The final classification is determined using a decision-rule-based evaluation mechanism designed to ensure consistent and reproducible labeling of generated test cases. This step explicitly separates semantic relevance from functional correctness and test completeness, thereby avoiding ambiguity between *related* and *partial pass* outcomes.

Let r denote a software requirement and t denote the generated test case. Let $\text{sim}(r, t) \in [0, 1]$ represent the semantic similarity between r and t , computed using embedding-based similarity. Let $f(t) \in \{0, 1\}$ indicate functional correctness, and let $c(t) \in \{0, 1\}$ indicate completeness of the required test elements.

Let θ_{sem} denote the semantic similarity threshold required for functional evaluation, and let θ_{rel} denote the minimum similarity threshold for topical relevance, where $\theta_{\text{rel}} < \theta_{\text{sem}}$.

The incorporation of partial correctness reflects the graded semantic nature of acceptance testing evaluation. Unlike binary classification tasks, acceptance test case alignment often exists on a continuum between complete correctness and structural relevance. To operationalize this evaluation in a reproducible manner, semantic similarity is quantified using embedding-based cosine similarity, and classification outcomes are determined through explicitly defined decision rules with fixed thresholds.

Partially correct outputs are identified when similarity scores fall within a predefined intermediate range, ensuring that grading is governed by deterministic criteria rather than subjective interpretation. This approach preserves the fundamental interpretation of precision (proportion of semantically aligned generated outputs) and recall/coverage (proportion of gold-standard criteria addressed), while allowing finer sensitivity to structural and semantic proximity.

Conceptually, this formulation aligns with graded relevance evaluation in information retrieval, where cosine similarity and ranking thresholds determine relevance levels, and with partial credit assessment frameworks in software engineering evaluation contexts.

Based on these definitions, the evaluation label $\text{FinalScore}(t)$ is determined according to the following decision rules:

$$\begin{aligned}
 &\text{FinalScore}(t) \\
 &= \begin{cases} \text{Pass,} & \text{if } \text{sim}(r, t) \geq \theta_{\text{sem}} \\ & \wedge f(t) = 1 \wedge c(t) = 1 \\ \text{PartialPass,} & \text{if } \text{sim}(r, t) \geq \theta_{\text{sem}} \\ & \wedge f(t) = 1 \wedge c(t) = 0 \\ \text{Related,} & \text{if } \theta_{\text{rel}} \leq \text{sim}(r, t) < \theta_{\text{sem}} \\ & \wedge f(t) = 0 \\ \text{Not Related,} & \text{if } \text{sim}(r, t) < \theta_{\text{rel}}. \end{cases}
 \end{aligned} \tag{13}$$

In this study, the thresholds were instantiated as $\theta_{\text{sem}} = 0.85$ and $\theta_{\text{rel}} = 0.50$, determined through empirical calibration to balance sensitivity and specificity across models.

For clarity, Table 4 provides an interpretable representation of the decision rules defined in Equation 13, using the same notation and threshold values.

TABLE 4. Decision-Rule–Based evaluation criteria for test case generation.

Category	Semantic Relevance $\text{sim}(r, t)$	Functional Correctness $f(t)$	Completeness $c(t)$	Decision Rule
Pass	$\text{sim}(r, t) \geq \theta_{\text{sem}}$	$f(t) = 1$	$c(t) = 1$	Fully satisfies semantic, functional, and completeness constraints.
Partial Pass	$\text{sim}(r, t) \geq \theta_{\text{sem}}$	$f(t) = 1$	$c(t) = 0$	Functionally correct but incomplete.
Related	$\theta_{\text{rel}} \leq \text{sim}(r, t) < \theta_{\text{sem}}$	$f(t) = 0$	$c(t) = 0$	Semantically relevant but functionally incorrect.
Not Related	$\text{sim}(r, t) < \theta_{\text{rel}}$	$f(t) = 0$	–	No semantic or functional alignment.

The overall procedure is summarized in Algorithm 2.

Note: Semantic similarity is estimated using the directional partially weighted ensemble, and final labels are determined by explicit decision rules.

4) CONFIDENCE INTERVALS

Confidence intervals (95% CI) have been added to all primary performance tables to reflect variability across five independent runs ($n = 5$). In addition to reporting mean values, interval estimates are provided to quantify statistical stability under stochastic LLM-based generation.

Given the stochastic behavior of LLM-based generation, each configuration was executed in five independent runs per requirement. The 95% confidence intervals (CI) [60], [61] were computed using the Student's t -distribution:

$$\text{CI} = \bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}} \quad (14)$$

where $\bar{x}$ denotes the mean coverage across runs, s denotes the sample standard deviation, and $n = 5$ represents the number of independent executions.

Since coverage is bounded between 0 and 100 percent, upper bounds exceeding 100 percent were truncated accordingly.

C. EXPERIMENTAL PROCEDURE

The performance of the proposed framework was evaluated through a two-phase experiment that involved training and testing. In the training phase, requirements 1–8 were used to calculate the weights. In the testing phase, requirements 9–16 were applied to assess the applicability and robustness of the framework to handling new requirements.

In the training phase, the framework was applied to Requirements 1–8. The following example experiment helps illustrate this method. For example, Requirement 8 (REQ8) specifies that reward points are approved upon the customer confirming receipt of the goods. The confirmation date is defined as the first day of the 180-day validity period. If a customer attempts to redeem points after this period, the system must reject the transaction. The example results focus on the logic condition and the expected outcome. The date period was not explicitly represented in the prompt.

REQ8's gold standard includes four illustrative test situations, detailed as follows: In the first case, the customer confirms receipt of the product, the reward points are less than 180 days old, they are redeemed as a discount, and the expected outcome is that the discount is applied successfully.

In the second case, the reward points are exactly 180 days old and are redeemed as a discount; the system should also successfully apply the discount. In the third case, the reward points are more than 180 days old and are redeemed as a discount; the expected result is that the redemption is rejected and no discount is granted. Finally, in the fourth case, the reward points are older than 180 days, but no redemption is attempted, and the transaction should be completed without applying a discount.

Once executed, each requirement in the training dataset was automatically generated for the test cases using the RAG approach. The model results were then manually mapped to the gold standard by the researchers. If the model accurately provided REQ8-TC01, the result was categorized as a true positive (TP). If the result indicates that the condition is met to approximately 50%, it is designated as a partial true positive (TP Partial). In certain instances, the model may produce results that do not precisely align with the functional test cases REQ8-TC01 to REQ8-TC04. These can be classified as related functional test cases or non-functional test cases, and the responses are allocated to the corresponding category.

Subsequently, the total scores for each classification are reported, which include TN, TP, TN (partial), TP (partial), FN, FP, related, and unrelated. Subsequently, these results were used to calculate key evaluation metrics, including test case coverage, accuracy, precision, recall, and F1-score.

The directional partially weighted ensemble model further integrates the predictions from multiple LLMs. For example, in the REQ8-TC02 result, each model yields a weighted score according to its classification performance: the true positive (TP) of Claude is scaled by its corresponding weight, the TP of GPT-4o is multiplied by its weight, and the TP of Gemini is multiplied in the same manner. The final ensemble score is derived by aggregating these weighted contributions. The resulting Final Score is interpreted as follows: a high score (≥ 0.85) indicates *pass/related*, a medium score (0.50–0.84) indicates partial pass/partially related, and a low score (< 0.50) indicates *fail/not related*. This categorization supports a more robust evaluation of the model agreement and reliability.

These weighted results, aggregated across all test cases, are summarized in Table 5 and Table 6. Subsequently, all requirements were assessed using eight text-generative AI tools, producing test cases and calculating the test coverage, accuracy, precision, recall, and F1-score for each iteration. This procedure was executed five times, after which a majority voting scheme reviewed the metrics. The five best

TABLE 5. Example of directional partially performance for Qwen-3 coder ensemble.

Test Case	Test Cases Detail	Expected Result	5-fold cross validation					Qwen3-coder
			fold (1)	fold (2)	fold (3)	fold (4)	fold (5)	Majority Vote
Functional Test Cases (Gold Standard Mapping)								
REQ8-TC01	The customer confirms receipt of the product, the reward points are less than 180 days old, and the points are redeemed as a discount.	Discount applied successfully.	TP	TP	TP	TP	TP	TP
REQ8-TC02	The customer confirms receipt of the product, the reward points are exactly 180 days old, and the points are redeemed as a discount.	Discount applied successfully.	FN	TP	Partial	TP	TP	TP
REQ8-TC03	The customer confirms receipt of the product, the reward points are more than 180 days old, and the points are redeemed as a discount.	Redemption rejected; discount not applied.	TN	TN	TN	TN	TN	TN
REQ8-TC04	The customer confirms receipt of the product, the reward points are more than 180 days old, and no points are redeemed for a discount.	Transaction completed without discount.	FP	TN	FP	FP	FP	FP
Other Model Outputs (Not Included in Gold Standard)								
REQ8-TC05	Others	Related	Related	Related	Related	Related	-	Related
REQ8-TC06	Non-Functional Testing	Related	-	-	-	-	-	-
REQ8-TC07	Not Related	Not Related	Not Related	Not Related	Not Related	-	-	-

TABLE 6. Example of weight ensemble score calculation.

Test Case	Expected Result	Claude	GPT-4	Gemini	Qwen	Mistral	3 Ensemble	4 Ensemble	5 Ensemble
REQ8-TC01	Discount applied Successfully	TP (1.0)	TP (1.0)	TP (1.0)	TP (1.0)	TP (1.0)	1.00=Pass	0.76=Partial	1.00=Pass
REQ8-TC02	Discount applied Successfully	TP (1.0)	FN (0.0)	TP (1.0)	TP (1.0)	TP (1.0)	0.74=Partial	0.51=Partial	0.84=Partial
REQ8-TC03	Rejected	TN (1.0)	TN (1.0)	TN (1.0)	TN (1.0)	TN (1.0)	1.00=Pass	0.76=Partial	1.00=Pass
REQ8-TC04	No discount	TN (1.0)	TN (1.0)	FP (0.0)	FP (0.0)	FP (0.0)	0.66=Partial	0.51=Partial	0.39=Fail
Coverage (%)		100	75	75	75	75	100	100	75
Accuracy		1.00	0.75	0.75	0.75	0.75	—		
Precision		1.00	1.00	0.67	0.67	0.67	—		
Recall		1.00	1.00	1.00	1.00	1.00	—		
F1-Score		1.00	1.00	0.80	0.80	0.80	—		

models were selected for the directional partially weighted ensemble based on test coverage, with matches sorted by average F1-score and accuracy.

The five best models for the weighted voting ensemble were selected based on test coverage, with matches ranked by average F1-score and accuracy. As shown in Table 7, Claude-Sonnet-4 achieved the highest average coverage of 91.32% (95% CI: [70.35%, 100.00%]), followed by GPT-4o at 88.89% (95% CI: [62.28%, 100.00%]) and Gemini-2.5-Flash at 88.54% (95% CI: [68.64%, 100.00%]). Qwen3-Coder and Mistral-Small-3.2 both achieved 85.94% coverage (95% CI: [60.44%, 100.00%]). The reported confidence intervals reflect variability across requirements in the training set.

Taking the average F1-score and accuracy (both 0.86) as additional selection criteria, these models were included in the ensemble construction. The top five models—Claude-Sonnet-4, GPT-4o, Gemini-2.5-Flash, Qwen3-Coder, and Mistral-Small-3.2—were subsequently used to construct ensembles of three, four, and five models, respectively. The weights were allocated proportionally, and a directional partially weighted ensemble algorithm was applied to compute the final score for each test case.

The weights of the three-model ensemble are 0.393 for Claude-Sonnet-4, 0.264 for GPT-4o, and 0.343 for Gemini-2.5-Flash. For the four-model ensemble, the weights were 0.292 for Claude-Sonnet-4, 0.196 for GPT-4o, 0.254 for Gemini-2.5-Flash, and 0.257 for Qwen3-Coder. The weights

TABLE 7. Individual LLM performance per requirement (Test Coverage, Accuracy, Precision, Recall, F1-Score) for Training Set.

LLM	Req.1	Req.2	Req.3	Req.4	Req.5	Req.6	Req.7	Req.8	Avg. Coverage	Avg(F1,Acc)	Std	95% CI
Claude-Sonnet-4												
Coverage (%)	100	75	100	56	100	100	100	100	91.32	0.98	16.89	[70.35, 100.00]
Accuracy	1.00	1.00	1.00	0.83	1.00	1.00	1.00	1.00	0.95	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	-	-	-
Recall	0.50	0.50	0.75	0.80	0.67	0.50	0.67	0.67	1.00	-	-	-
F1-Score	0.67	0.67	0.86	0.89	0.80	0.67	0.80	0.80	1.00	-	-	-
GPT-4o												
Coverage (%)	100	100	100	44	100	100	100	67	88.89	0.66	21.41	[62.28, 100.00]
Accuracy	1.00	1.00	1.00	0.67	1.00	1.00	1.00	0.80	0.51	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.68	-	-	-
Recall	1.00	1.00	0.75	0.75	0.67	0.67	1.00	0.50	1.00	-	-	-
F1-Score	1.00	1.00	0.86	0.86	0.80	0.80	1.00	0.57	0.80	-	-	-
Gemini 2.5 Flash												
Coverage (%)	100	100	100	66.67	66.67	100	100	75	88.54	0.85	16.02	[68.64, 100.00]
Accuracy	1.00	1.00	1.00	0.75	0.67	1.00	1.00	0.75	0.90	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.94	-	-	-
Recall	1.00	1.00	1.00	0.83	0.50	1.00	0.67	1.00	0.74	-	-	-
F1-Score	1.00	1.00	1.00	0.91	0.67	1.00	0.80	0.80	0.80	-	-	-
Qwen3-Coder												
Coverage (%)	100	100	100	62.50	50	100	100	75	85.94	0.86	20.53	[60.44, 100.00]
Accuracy	1.00	1.00	1.00	1.00	0.50	1.00	1.00	0.75	0.91	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.94	-	-	-
Recall	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.75	-	-	-
F1-Score	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.80	0.81	-	-	-
Mistral Small 3.2												
Coverage (%)	100	100	100	62.50	50	100	100	75	85.94	0.86	20.53	[60.44, 100.00]
Accuracy	1.00	1.00	1.00	1.00	0.50	1.00	1.00	0.75	0.91	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.94	-	-	-
Recall	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.75	-	-	-
F1-Score	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.80	0.81	-	-	-
GLM-4.5v												
Coverage (%)	100	100	100	37.50	50	100	100	75	82.81	0.92	25.82	[50.74, 100.00]
Accuracy	1.00	1.00	1.00	0.75	0.50	1.00	1.00	0.75	0.91	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.97	-	-	-
Recall	1.00	1.00	1.00	0.75	1.00	1.00	1.00	1.00	0.93	-	-	-
F1-Score	1.00	1.00	1.00	0.86	1.00	1.00	1.00	0.80	0.93	-	-	-
Perplexity Sonar Pro												
Coverage (%)	100	100	100	50	50	100	100	75	84.38	0.92	22.90	[55.94, 100.00]
Accuracy	1.00	1.00	1.00	0.88	0.75	1.00	1.00	0.75	0.90	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.96	-	-	-
Recall	1.00	1.00	1.00	0.88	0.33	1.00	1.00	1.00	0.93	-	-	-
F1-Score	1.00	1.00	1.00	0.93	0.50	1.00	1.00	0.80	0.93	-	-	-
DeepSeek R1												
Coverage (%)	100	100	100	37.50	50	100	100	75	82.81	0.86	25.82	[50.74, 100.00]
Accuracy	1.00	1.00	1.00	0.89	0.50	1.00	1.00	0.75	0.92	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.95	-	-	-
Recall	1.00	1.00	1.00	0.88	1.00	1.00	1.00	1.00	0.73	-	-	-
F1-Score	1.00	1.00	1.00	0.93	1.00	1.00	1.00	0.80	0.80	-	-	-

Note: Since coverage is bounded between 0 and 100, the upper bound of the 95% confidence interval was truncated at 100%.

for the five-model ensemble were 0.232 for Claude-Sonnet-4, 0.156 for GPT-4o, 0.202 for Gemini-2.5-Flash, 0.205 for Qwen3-Coder, and 0.205 for Mistral-Small-3.2.

Table 8 summarizes the performance of five LLM models across requirements REQ9–REQ16, including the corresponding 95% confidence intervals (CI) for average coverage. Overall, all models achieved high test coverage and strong performance in accuracy, precision, recall, and F1-score, with moderate variability across individual requirements as reflected in the reported standard deviations and confidence intervals.

Claude-Sonnet-4 achieved an average coverage of 96.35% (95% CI: [87.86%, 100.00%]) with an average F1-score and accuracy of 0.92. Minor performance degradation was observed in recall for REQ10 (0.80) and REQ12 (0.67), contributing to the observed variability. GPT-4o achieved 95.42% coverage (95% CI: [84.83%, 100.00%]) and the

highest overall accuracy (0.99), maintaining strong performance across most requirements, although precision and recall declined for REQ13 and REQ14.

Gemini-2.5-Flash demonstrated 97.73% coverage (95% CI: [89.75%, 100.00%]), with performance reductions observed in REQ16 (precision 0.75, F1-score 0.86). Qwen3-Coder achieved the highest average coverage at 98.44% (95% CI: [92.95%, 100.00%]) and maintained consistently high F1-scores across requirements, indicating stable performance with comparatively lower variance. Mistral-Small-3.2 achieved 95.31% coverage (95% CI: [83.76%, 100.00%]); however, precision (0.50) and F1-score (0.67) were substantially lower for REQ11, contributing to its higher dispersion.

A directional partially weighted ensemble method was employed to mitigate requirement-level variability by leveraging the strongest-performing models. The weights of

TABLE 8. Individual LLM performance per requirement (Test Coverage, Accuracy, Precision, Recall, F1-Score) for Testing Set.

LLM	Req.9	Req.10	Req.11	Req.12	Req.13	Req.14	Req.15	Req.16	Avg. Coverage	Avg(F1,Acc)	Std	95% CI
Claude-Sonnet-4												
Coverage (%)	100	100	100	100	100	83.33	87.50	100	96.35	0.92	6.84	[87.86, 100.00]
Accuracy	1.00	1.00	1.00	1.00	1.00	0.83	0.88	1.00	0.97	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	0.83	1.00	1.00	0.97	-	-	-
Recall	1.00	0.80	1.00	0.67	0.80	0.60	0.86	1.00	0.79	-	-	-
F1-Score	1.00	0.89	1.00	0.80	0.89	0.70	0.92	1.00	0.86	-	-	-
GPT-4o												
Coverage (%)	100	100	100	100	80	83.33	100	100	95.42	0.95	8.53	[84.83, 100.00]
Accuracy	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.99	-	-	-
Precision	1.00	1.00	1.00	1.00	0.80	0.80	1.00	1.00	0.94	-	-	-
Recall	1.00	1.00	1.00	1.00	1.00	0.75	0.88	1.00	0.89	-	-	-
F1-Score	1.00	1.00	1.00	1.00	0.89	0.77	0.93	1.00	0.91	-	-	-
Gemini 2.5 Flash												
Coverage (%)	100	100	100	100	100	100	100	81.82	97.73	0.95	6.43	[89.75, 100.00]
Accuracy	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.90	0.99	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.75	0.96	-	-	-
Recall	1.00	1.00	1.00	1.00	0.80	0.60	0.88	1.00	0.87	-	-	-
F1-Score	1.00	1.00	1.00	1.00	0.89	0.75	0.93	0.86	0.90	-	-	-
Qwen3-Coder												
Coverage (%)	100	100	100	100	100	100	87.50	100	98.44	0.94	4.42	[92.95, 100.00]
Accuracy	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	-	-	-
Precision	1.00	1.00	1.00	1.00	1.00	1.00	0.88	1.00	0.93	-	-	-
Recall	1.00	1.00	1.00	1.00	0.80	0.60	1.00	1.00	0.86	-	-	-
F1-Score	1.00	1.00	1.00	1.00	0.89	0.75	0.93	1.00	0.88	-	-	-
Mistral Small 3.2												
Coverage (%)	100	100	75	100	100	100	87.50	100	95.31	0.91	9.30	[83.76, 100.00]
Accuracy	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.98	-	-	-
Precision	1.00	1.00	0.50	1.00	1.00	1.00	0.88	1.00	0.89	-	-	-
Recall	1.00	1.00	1.00	1.00	0.80	0.60	1.00	1.00	0.83	-	-	-
F1-Score	1.00	1.00	0.67	1.00	0.89	0.75	0.93	1.00	0.84	-	-	-

Note: Since coverage is bounded between 0 and 100, the upper bound of the 95% confidence interval was truncated at 100%.

the three-model ensemble were 0.326 for Claude-Sonnet-4, 0.338 for GPT-4o, and 0.336 for Gemini-2.5-Flash. For the four-model ensemble, the weights were 0.244 for Claude-Sonnet-4, 0.253 for GPT-4o, 0.252 for Gemini-2.5-Flash, and 0.251 for Qwen3-Coder. The weights for the five-model ensemble were 0.196 for Claude-Sonnet-4, 0.204 for GPT-4o, 0.203 for Gemini-2.5-Flash, 0.202 for Qwen3-Coder, and 0.195 for Mistral-Small-3.2.

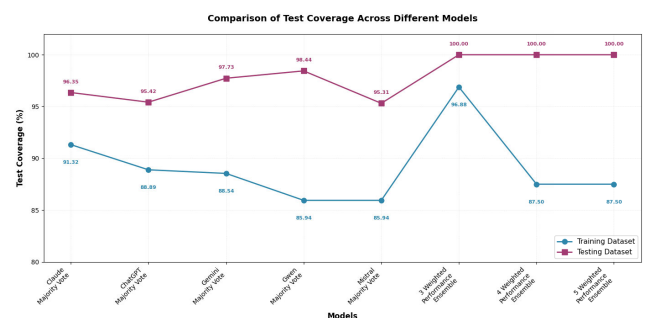
Weights were allocated proportionally based on each model's combined F1-score and accuracy, ensuring balanced contributions while reducing the impact of requirement-specific performance fluctuations.

V. EXPERIMENT RESULTS

The experimental evaluation, across both the training and testing datasets, revealed substantial discrepancies in performance among models, ensemble configurations, and testing methodologies.

A. TRAINING DATASET PERFORMANCE

The training dataset consisted of eight requirements (REQ1–REQ8) and 31 test cases. The experiment evaluated five AI models, Claude, ChatGPT, Gemini, Qwen, and Mistral, across four software testing techniques: Boundary Value Analysis (BVA), Equivalence Partitioning (EP), Use Case (UC), and State Transition (ST). Then, the directional

**FIGURE 4.** Percentage of test coverage comparison.

partially weighted ensembles of three, four, and five models were compared. All reported confidence intervals reflect variability across five independent executions per configuration.

The three-model ensemble demonstrated higher rank performance, achieving an average coverage of 96.9%, earning six perfect scores out of eight criteria, and achieving a minimum coverage of 75% across every requirement, as illustrated in Fig. 4. In contrast, the four- and five-model ensembles showed decreased average coverage (87.5%), minor performance in REQ5 and REQ8, and increased inconstancy (standard deviation: 18.9 compared to 8.8 for the three-model ensemble).

Performance varies according to the requirements. REQ1, REQ3, REQ6–REQ7 (utilizing BVA) and REQ2 (employing

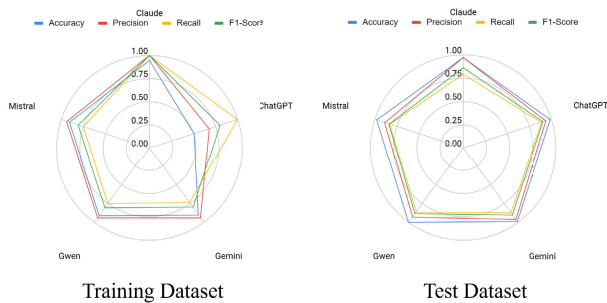


FIGURE 5. Performance result comparison.

EP) were consistently accurately predicted. In contrast, REQ4 (BVA with two conditions), REQ5 (UC), and REQ8 (ST) presented greater difficulty. Although the four-model ensemble excelled significantly with 95.9% coverage in the test data scenario, the smaller ensembles show reduced distortion and achieve the highest balance of precision and resilience to threshold sensitivity, and thus achieved the most effective strategy.

B. TEST DATASET PERFORMANCE

The test dataset (REQ9–REQ16) had 54 test case responses. These were sourced from actual projects, Kaggle, and the ISTQB materials. Weighted ensemble models were used for evaluation.

Here, the four- and five-model ensembles performed better. They achieved average coverages of 95.9% and 95.8%, respectively. The three-model ensemble achieved 89.6%.

REQ10, REQ12, and REQ13 showed significant improvement, while REQ14 remained challenging across all ensembles (coverage 70 – 77%). The results indicate that larger ensembles achieve higher rankings on testing data, especially when evaluating equivalence partitioning and use case-based criteria, although certain complex scenarios still limit performance gains.

Although mean coverage differs across configurations, overlapping 95% CI ranges suggest that certain improvements should be interpreted cautiously.

VI. DISCUSSION

A. MODEL-SPECIFIC STRENGTHS AND WEAKNESSES

The performance metrics in Fig. 5 show distinct characteristics in both training and testing datasets.

- 1) **Training dataset:** Claude's majority vote demonstrated high precision, recall, and F1-score (1.00), indicating a robust match. Gemini, Qwen, and Mistral had balanced performance, with accuracies of about 0.90 and F1-scores ranging from 0.80 to 0.81. ChatGPT performed inadequately (accuracy 0.51, F1-score 0.80), suggesting a higher chance of underfitting.
- 2) **Test dataset:** All models achieved high accuracy (0.97–1.00). However, recall and F1-score differed. ChatGPT had a high recall (0.89) and an F1-score of

0.91, indicating a good balance. Gemini and Qwen performed similarly (F1 0.88–0.90). Claude had high accuracy (0.97) but lower recall (0.79), suggesting fewer positive cases. Mistral had weaker results (recall 0.83, F1 0.84).

Qwen and Mistral exhibited the highest recall (0.92) and competitive F1-scores (0.94 and 0.93), making them more accurate in identifying true positives. Claude emphasized precision (0.98) over recall (0.74), whereas ChatGPT and Gemini achieved similar F1 Scores (0.90). Ensemble approaches and weighted voting effectively integrate complementary strengths and handle performance discrepancies, as shown by the findings.

B. TEST COVERAGE BY TECHNIQUE

Fig. 6 shows an overview of test coverage across different models and methods.

- **EP:** All models and ensembles consistently achieved 100% coverage.
- **BVA:** Individual models achieved 100% coverage; however, ensembles of four and five models decreased to 83.33%, suggesting complications associated with weighting.
- **Use Case (UC):** Coverage remained stable at 95.83% for ensemble techniques, with only slight variation among individual models; The mistral was marginally lower at 91.67%.
- **State Transition (ST):** Performance varied notably: Claude, ChatGPT, and the three-model ensemble achieved 100%, whereas Gemini, Qwen, and Mistral recorded 66.67%, 50%, and 50%, respectively. The four- and five-model ensembles improved coverage to 91.67%.

The results show that ensembles, especially the three-model weighted ensemble, improve robustness and reliability. However, there are trade-offs, including a minor reduction in UC coverage.

C. ENSEMBLE TRADE-OFFS

A comparative investigation of ensemble configurations reveals trade-offs:

- **Three-model ensemble:** Optimal overall performance (93.2%), minimal standard deviation (10.04), nine perfect scores, most balanced and stable.
- **Four-model ensemble:** Slightly reduced overall score (91.7%), nine flawless scores and comprehensive test data.
- **Five-model ensemble:** This achieved similar performance (91.6%) but less consistency (standard deviation 14.85), indicating reduced reliability.

Smaller ensembles yield a more favorable balance of accuracy and robustness by reducing internal disagreement among model predictions and minimizing the risk of overfitting to specific patterns in the dataset. These observations indicate that such compact ensembles can outperform larger ones.

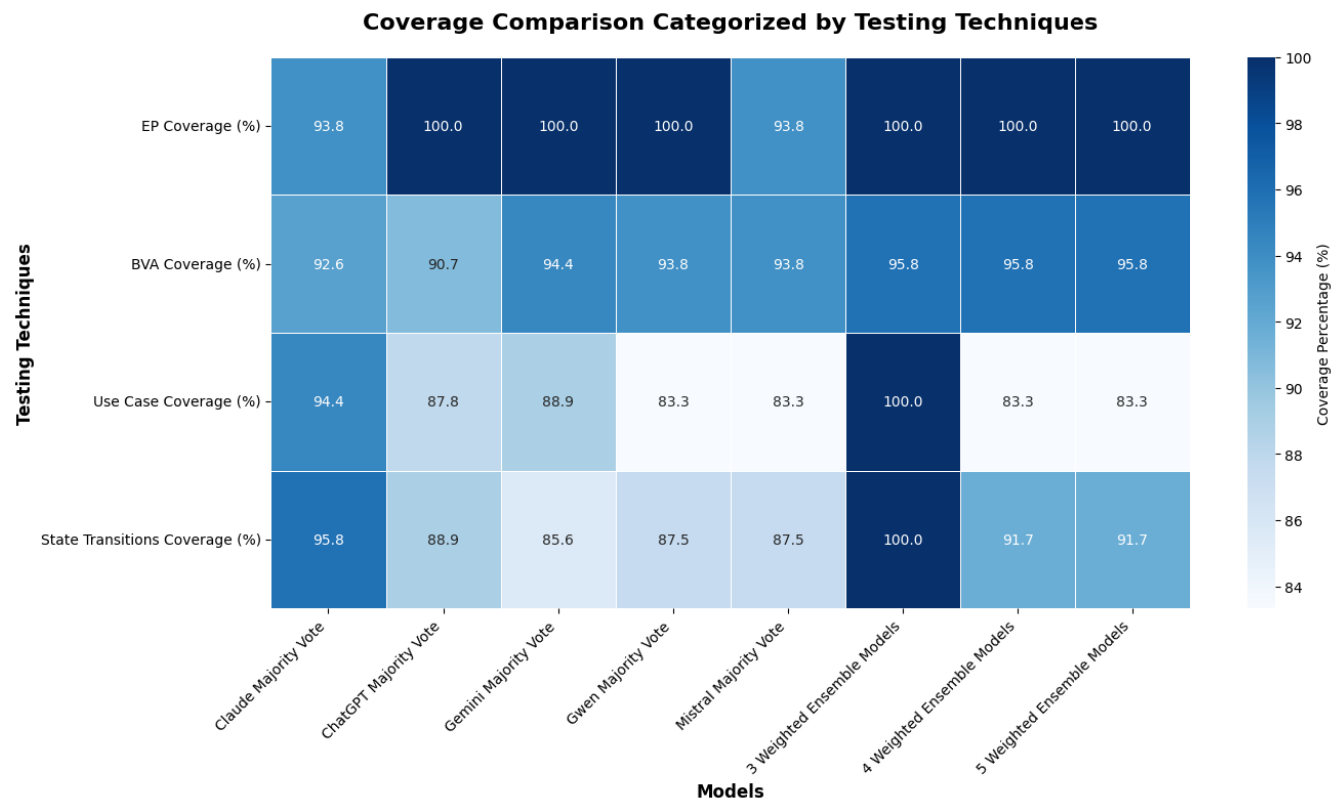


FIGURE 6. LLM-Driven automated software test case generation using a directional partially weighted ensemble with RAG.

In contrast, larger ensembles may introduce redundancy or conflicting outputs, thereby reducing the overall stability of the final decision.

Depending on the use case, the three-model ensemble shows more accuracy, consistency, and training effectiveness. The four-model ensemble has greater extension. Adding more models does not always improve performance. Larger ensembles offer benefits only in particular cases.

This effect is further explained by test distribution skew, under which a small subset of consistently strong models dominates the prevalent requirement patterns. In contrast, larger ensembles are more likely to incorporate weaker or misaligned models that amplify noise and internal disagreement.

D. EVALUATION OF WEIGHT ENSEMBLE

A comparative study was conducted on ensemble models with three, four, and five weights, tested across several datasets.

The training dataset shows that the three-model group had the highest coverage at 96.88%, exceeding the four- and five-model groups at 87.50

In the test data set, the three-model group got a perfect test coverage (100.00%).

Overall, the results indicate that three-model ensembles achieved the highest test coverage of 98.44 percent, whereas

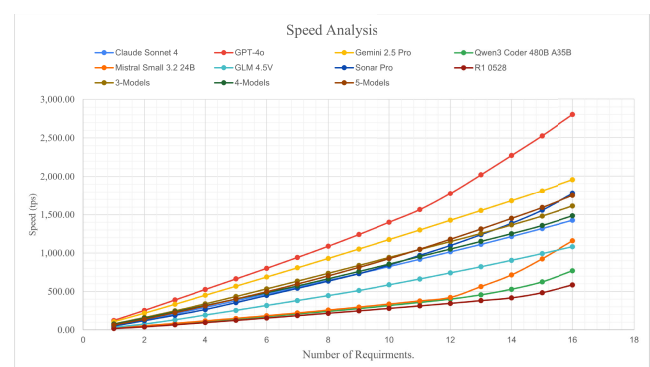


FIGURE 7. Speed analysis.

the four- and five-model ensembles achieved the same test coverage of 93.75 percent.

These results suggest that smaller weighted-vote ensembles yield more consistent results because weighted-vote mechanisms enhance each model's predictive power without averaging too much, thereby preserving each model's contributions. Conversely, larger weighted ensembles may experience lower returns, as overlapping model behaviors reduce marginal gains in accuracy. Consequently, smaller ensembles maintain a superior balance of precision, interpretability, and computational efficiency.

The relatively narrow 95% confidence intervals observed for the three-model ensemble indicate stable performance

AS A Professional software tester,
I WANT to generate test cases covering both functional and error scenarios from this condition: “\${req_condition}”,
AND I WANT the test cases to be relevant to the requirement type, whether it involves business rules, state transitions, input validation, or system behavior,
AND: Present ONLY the test cases in a table format with the following columns: “Test Case ID”, “Test Condition”, “Real Example Data”, and “Expected Result”.
SO THAT consumers, developers, and QA teams can verify the correctness of the system under various conditions.
AND Do NOT include any introductory or concluding text.
AND Relevant Information Extracted from the Document: “\${information}”

(a) User Story Prompt Template

ROLE: Professional software tester,
INSTRUCTION: Generate test cases covering both functional and error scenarios from this condition: “\${req_condition}”,
CONTEXT: Relevant Information Extracted from the Document: “\${information}”
EXAMPLE: Present ONLY the test cases in a table format with the following columns: “Test Case ID”, “Test Condition”, “Real Example Data”, and “Expected Result”.
EXCLUSION: exclude any introductory or concluding text.

(b) RICEE Prompt Template

PERSONA: Professional software tester,
AIM: Generate test cases covering both functional and error scenarios from this condition: “\${req_condition}”,
RECIPIENT: consumers, developers, and QA teams
THEME: Use a formal and clear style and tone.
STRUCTURE: Present ONLY the test cases in a table format with the following columns: “Test Case ID”, “Test Condition”, “Real Example Data”, and “Expected Result”.
 Relevant Information Extracted from the Document: “\${information}”

(c) PARTS Prompt Template

FIGURE 8. Example of prompt templates.

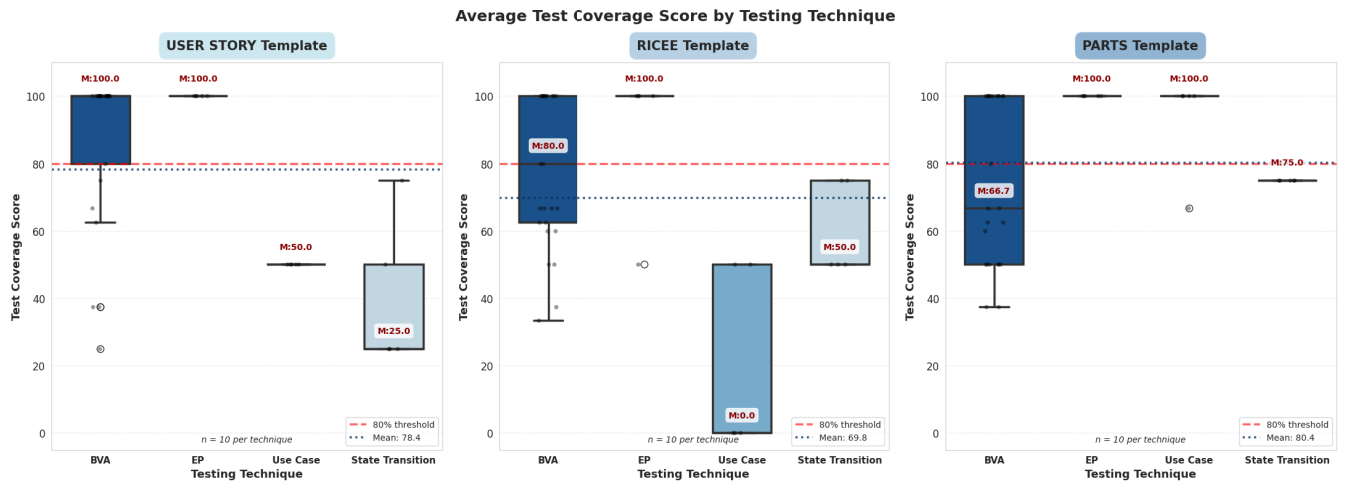


FIGURE 9. 3-Prompt templates test case coverage percentage comparison.

across repeated stochastic generations. In contrast, wider intervals in larger ensembles suggest increased variability due to internal prediction disagreement.

E. SPEED COMPLEXITY

Fig. 7 illustrates that the time complexity across all evaluated tools follows a convex upward trend, indicating a linear scaling behavior of the throughput rate $\text{Big-O} = O(n)$, linear growth, $R^2 \approx 0.99$). Among the models examined, GPT-4o exhibits the highest scaling efficiency, while R1-0528 demonstrates the lowest. The results suggest that the

use of 3 to 5 ensemble models strikes the optimal balance between system performance and computational overhead, particularly under linear scalability ($O(n)$). The 5-weighted model ensemble achieves the highest throughput among the various ensemble groups. Additionally, the 3- and 4-weight ensemble models are well-suited to resource-constrained environments with stringent speed requirements.

F. ARCHITECTURAL SCALABILITY

Beyond computational efficiency, scalability in software testing practice extends beyond dataset size or execution

volume to include methodological adaptability across heterogeneous testing paradigms. Industrial systems differ in domain characteristics, architectural constraints, regulatory demands, and risk profiles; consequently, testing strategies vary substantially. Boundary Value Analysis, Equivalence Partitioning, Use Case testing, and State Transition testing embody distinct reasoning logics and validation objectives.

The proposed framework demonstrates methodological scalability by integrating multiple closed box testing techniques within a unified Retrieval-Augmented Generation (RAG) and directional partially weighted ensemble architecture. Rather than being optimized for a single test-generation pattern, the system accommodates diverse requirements structures and evaluation logics without architectural modification.

In practical industrial settings, testing knowledge evolves incrementally. As new requirements and domain artifacts are incorporated into the retrieval knowledge base, contextual grounding expands without structural redesign, enabling gradual scaling toward more complex systems while preserving architectural coherence.

From a reproducibility standpoint, the framework enforces structural reproducibility and procedural consistency through explicit semantic thresholds, formalized ensemble weighting, and rule-based decision mechanisms. Although individual LLM outputs remain stochastic, aggregation and evaluation follow deterministic rules, supporting systematic replication across datasets and testing domains.

Accordingly, scalability in this study should be interpreted as architectural and methodological scalability rather than empirical validation on large-scale industrial deployments. Large-scale production validation remains an avenue for future work.

While the proposed framework demonstrates strong performance within the defined experimental scope, its findings should be interpreted within the current evaluation context. The benchmark dataset reflects a newly constructed setting in an emerging research area where no widely established benchmark yet exists for RAG-enhanced, ensemble-based acceptance test case generation. Although the selected requirements were designed to capture diverse testing methodologies and industry-relevant scenarios, continued community-driven benchmarking efforts would further enhance comparative assessment. The dataset scale (16 requirements, 76 test cases) supports controlled and transparent evaluation; nevertheless, extending validation across additional domains and system complexities would provide broader empirical confirmation. Furthermore, the partial correctness evaluation employs calibrated semantic similarity thresholds grounded in empirical testing and expert validation. Sensitivity analysis indicates stable behavior within a reasonable operational range, though future adaptive mechanisms may further enhance cross-domain flexibility. Future research will focus on expanding benchmark diversity, promoting reproducible public evaluation datasets, and investigating adaptive or data-driven threshold optimization

mechanisms to enhance robustness, scalability, and generalizability across varied industrial contexts.

VII. CONCLUSION

This experiment presents automated software test case generation using Weighted-Vote Ensemble Models with retrieval-agnostic Generation (RAG), which enhances accuracy, coverage, and robustness by combining outputs from multiple LLMs. The dataset consists of 16 requirements (REQ1–REQ16) and 76 test cases based on real-world projects, ISTQB, and Kaggle. Software testing techniques were used, including Boundary Value Analysis (BVA), Equivalence Partitioning (EP), Use Case (UC), and State Transition (ST). This study examined five generative AI models: anthropic/claude-sonnet-4, openai/gpt-4o, google/gemini-2.5-pro, qwen/qwen3-coder, and mistralai/mistral-small-3.2-24b-instruct. A directional partially weighted ensemble was calculated based on the majority vote from the top-performing training dataset and on comparative performance across the training and testing datasets.

For individual LLMs, Claude performed the best, achieving the highest test coverage of 93.84% with an accuracy score of 0.97. In contrast, Mistral performed the worst, with 90.63% test coverage but the same accuracy score of 0.97. When using ensembles, the three-model weighted ensemble achieved the greatest improvement, increasing the coverage by +4.60%. The ensembles of four or five models provided only marginal gains, with a slight decrease of −0.09% compared to Claude. In general, across both training and test datasets, the three-model ensemble consistently outperformed individual models, improving test coverage by +5.56% and training coverage by +1.56%.

A comprehensive experiment showed distinct trade-offs between accuracy, consistency, and generalization. The three models achieved the highest test coverage of 96.9%, with a standard deviation of 8.8. The four-model ensemble achieved the highest test coverage of 95.9% on the test dataset, a slight improvement over the five-model ensemble's 95.8%. In contrast, the three-model ensemble dropped to 89.6%. The three-model ensemble achieved an optimal balance between accuracy and consistency, with an overall score of 93.2%. However, the five-model ensemble showed no notable benefit over the three- or four-model configuration, indicating diminishing returns. In contrast, the four-model ensemble demonstrated superior generalization to novel data.

An analysis of coverage across various software testing types indicates that the BVA had nearly flawless results (training, 91.0%; test, 100%). EP showed strong performance, although a minor decrease was observed from training (100%) to testing (96.6%). UC showed the most significant improvement (training: 70.0%, test: 94.7%), while ST also improved considerably (training: 80.0%, test: 90.9%). These results indicate that, although UC and ST pose difficulties during training, the ensembles achieved higher rankings in generating test cases.

Requirement-level analyses revealed the most challenging scenarios. In the training dataset, REQ4 (BVA2), REQ5 (UC), and REQ8 (ST) were the least effective, with a minimum score of 37.5%. In the test data set, REQ15 and REQ16 (ST), REQ13 and REQ14 (UC), and REQ11 (EP) remain challenging, with a coverage of approximately 69%, highlighting ongoing difficulties with sequential and complex requirements.

Furthermore, the speed analyses indicate that all the models assessed exhibit linear scaling in throughput ($O(n)$), reflecting proportional growth with task volume. The individual results indicate that GPT-4o achieves the highest time efficiency. The 5-model ensemble achieves an optimal balance in throughput under linear-scaling conditions. However, the 3-, 4-, and 5-weighted ensemble models rank moderately in terms of speed.

In conclusion, this study demonstrates that weighted-vote ensemble strategies combined with retrieval-agnostic generation provide a robust and scalable approach to automated software test case generation. Claude emerged as the most reliable individual model. At the same time, three-model ensembles achieved the best balance between accuracy and stability, and four-model ensembles offered superior generalization on test data. Performance was strongly influenced by coverage type, with BVA showing consistent reliability, whereas UC and ST remained challenging due to their sequential and contextual complexity. These findings highlight the importance of selecting ensemble configurations based on specific testing objectives: smaller ensembles favor consistency and efficiency, whereas larger ensembles enhance generalization while maintaining linear speed scalability.

Based on these findings, the semantic thresholds were empirically calibrated and validated by experts in this study; however, research will explore adaptive thresholding strategies based on validation-set distribution modeling or reinforcement-based feedback mechanisms to further automate and generalize the evaluation process.

ACKNOWLEDGMENT

The Department of Computer Science, College of Computing, Khon Kaen University, Khon Kaen, Thailand, furnished support for the initiative.

The authors express their sincere appreciation to We Love Bug Company Ltd. and Siam Chamnankit Company Ltd. for their invaluable insights provided during the review process. They also acknowledge their gratitude to the two anonymous reviewers for their valuable insight, which considerably improved the quality of this work.

They further acknowledge the use of large language models and AI systems, including ChatGPT (OpenAI), Gemini (Google DeepMind), Claude (Anthropic), Qwen (Alibaba), GLM-4.5v (Zhipu AI), Perplexity AI, Sonar Pro, and DeepSeek R1, to assist in generating test cases for user acceptance testing plus English checking. In addition, they used ChatGPT (OpenAI) to improve the readability of the

manuscript. They carefully reviewed, validated, and revised all AI-assisted content to ensure accuracy and reliability.

AUTHORS CONTRIBUTIONS

The authors delineate their roles in this study as follows, Wantana Sisomboon wrote the main manuscript, including review, conceptualization, research, methodology, and analysis. Jakkrit Kaewyotha was responsible for research design, review, and analysis. Wararat Songpan was responsible for writing, conceptualization, research design, supervision, review, and analysis. All authors reviewed and approved the final version of the manuscript for publication.

APPENDIX PROMPT TEMPLATE ENGINEERING

Appendix presents a comparative study of three prompt-template patterns used to generate automated test cases from the training dataset (see Table 2).

- 1) **User Story:** The user story format uses the structure “As a [role], I want [goal], so that [purpose],” which is characteristic of user stories in Agile, as shown in Fig. 8a.

- 2) **RICEE:** The RICEE framework consists of six keys, as shown in Fig. 8b.

Role: The initial step is to define a specific role for the AI Text Generator to provide a specific language description, and the details of the AI’s output adhere to the desired style and substance.

Instructions: These instructions direct the AI Text Generator to provide detailed and comprehensive instructions to ensure that all necessary aspects are covered.

Context: Context is essential to guide AI to produce focused and relevant information.

Example: The examples set the standard for the expected quality and tone of output according to these guidelines.

Exception: Defining limits ensures that the AI Text Generator’s output remains focused and relevant, avoiding generic or irrelevant content.

- 3) **PARTS:** Google’s PARTS structure can be broken down into five key elements that enhance the effectiveness of prompt writing, as shown in Fig. 8c.

Persona: Identify the specific role or persona that the AI is supposed to perform.

Objective: The purpose is to provide a clear aim of AI-generated content.

Recipients: Specify the target audience for the AI’s output.

Theme: Identify the preferred style, tone, and other important parameters that should be present in the content.

Structure: Specify the appropriate format for AI-generated output, ensuring that it meets the intended use case.

By following these principles, researchers and practitioners can create prompts that produce precise, contextually relevant, and high-quality AI-generated content.

The experiment applied three prompt-template patterns to generate test cases for four software testing techniques, using eight requirements from the training dataset. Five LLM tools were used to evaluate performance. The resulting test-coverage comparison is shown in Fig. 9.

Overall, the PARTS template achieved the highest coverage across all four testing techniques. In particular, it reached 100% coverage for the Use Case technique, indicating a strong alignment between the PARTS structure and the generation of scenario-based tests. However, State Transition testing remained challenging across all prompt templates, suggesting that LLMs still struggle to capture complex state-dependent logic.

REFERENCES

- [1] K. Pugh, *Lean-Agile Acceptance Test-Driven Development: Better Software Through Collaboration*. Boston, MA, USA: Addison-Wesley, 2011.
- [2] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," 2023, *arXiv:2307.07221*.
- [3] W. Sisomboon, J. Kaewyotha, P. Dansakulcharoenkit, and W. Songpan, "AI-driven prompt templates for user acceptance test case generation," in *Data Science and Artificial Intelligence* (Communications in Computer and Information Science), vol. 2318. Singapore: Springer, 2025, pp. 78–92, doi: [10.1007/978-981-97-9793-6_6](https://doi.org/10.1007/978-981-97-9793-6_6).
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. nEURAL Inf. Process. Syst.*, 2020, pp. 1–16.
- [5] M. Arslan, H. Ghanem, S. Munawar, and C. Cruz, "A survey on RAG with LLMs," *Proc. Comput. Sci.*, vol. 246, pp. 3781–3790, Jan. 2024.
- [6] S. P. Bande, N. Vinutha, and P. Kumar, "Augmenting medical diagnostics with AI: A dual approach using RAG-based chatbots and nano GPT models," in *Proc. 6th Int. Conf. Recent Adv. Inf. Technol. (RAIT)*, Mar. 2025, pp. 1–6, doi: [10.1109/RAIT65068.2025.11088917](https://doi.org/10.1109/RAIT65068.2025.11088917).
- [7] A. Kulshreshtha, A. Choudhary, T. Taneja, and S. Verma, "Enhancing healthcare accessibility: A RAG-based medical chatbot using transformer models," in *Proc. Int. Conf. IT Innov. Knowl. Discovery (ITIKD)*, Apr. 2025, pp. 1–4, doi: [10.1109/ITIKD63574.2025.11005179](https://doi.org/10.1109/ITIKD63574.2025.11005179).
- [8] H. Wu and H. Wu, "Revolutionizing RAG with confliction detection: Transforming customer service chatbots with large language models," in *Proc. IEEE 4th Int. Conf. Comput. Mach. Intell. (ICMI)*, Apr. 2025, pp. 1–6, doi: [10.1109/ICMI65310.2025.11141256](https://doi.org/10.1109/ICMI65310.2025.11141256).
- [9] F. Pan, Q. Zhou, W. Guo, and H. Yang, "A survey on retrieval-augmented generation in applications of education and teaching," in *Proc. 7th Int. Conf. Comput. Sci. Technol. Educ. (CSTE)*, Apr. 2025, pp. 803–807, doi: [10.1109/CSTE64638.2025.11092120](https://doi.org/10.1109/CSTE64638.2025.11092120).
- [10] S. Selva Kumar, A. K. M. A. Khan, I. A. Banday, M. Gada, and V. V. Shanbhag, "Overcoming LLM challenges using RAG-driven precision in coffee leaf disease remediation," in *Proc. Int. Conf. Emerg. Technol. Comput. Sci. Interdiscipl. Appl. (ICETCS)*, Apr. 2024, pp. 1–6, doi: [10.1109/ICETCS61022.2024.10543859](https://doi.org/10.1109/ICETCS61022.2024.10543859).
- [11] R. Sheikh, D. Kumar, S. K. Singh, A. J. Hati, and J. U. Kidav, "AgroBuddy: Empowering Indian farmers through precision farming chatbot," in *Proc. 3rd Int. Conf. Commun., Secur., Artif. Intell. (ICCSAI)*, Apr. 2025, pp. 1627–1632, doi: [10.1109/ICCSAI64074.2025.11064149](https://doi.org/10.1109/ICCSAI64074.2025.11064149).
- [12] K. R. Raghi, K. Sudha, A. M. Sreeram, and S. Joshua, "Software development automation using generative AI," in *Proc. Int. Conf. Emerg. Res. Comput. Sci. (ICERCS)*, Dec. 2024, pp. 1–6, doi: [10.1109/ICERCS63125.2024.10894980](https://doi.org/10.1109/ICERCS63125.2024.10894980).
- [13] J. Shin, N. Shiri Harzevili, R. Aleithan, H. Hemmati, and S. Wang, "Retrieval-augmented test generation: How far are we?" 2024, *arXiv:2409.12682*.
- [14] Z. Zhang, X. Liu, Y. Lin, X. Gao, H. Sun, and Y. Yuan, "LLM-based unit test generation via property retrieval," 2024, *arXiv:2410.13542*.
- [15] S. M. Khandaker, F. Kifetew, D. Prandi, and A. Susi, "AugmenTest: Enhancing tests with LLM-driven oracles," in *Proc. IEEE Conf. Softw. Test. Verification Validation (ICST)*, Mar. 2025, pp. 279–289, doi: [10.1109/ICST62969.2025.10988926](https://doi.org/10.1109/ICST62969.2025.10988926).
- [16] S. Li, L. Stenzel, C. Eickhoff, and S. Ali Bahrainian, "Enhancing retrieval-augmented generation: A study of best practices," 2025, *arXiv:2501.07391*.
- [17] A. U. R. Butt, M. Asif, S. Ahmad, and U. Imdad, "An empirical study for adopting social computing in global software development," in *Proc. 7th Int. Conf. Softw. Comput. Appl.*, Feb. 2018, pp. 31–35, doi: [10.1145/3185089.3185105](https://doi.org/10.1145/3185089.3185105).
- [18] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, "Seven failure points when engineering a retrieval augmented generation system," 2024, *arXiv:2401.05856*.
- [19] M. Wortsman, G. Ilharco, S. Yitzhak Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, and L. Schmidt, "Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," 2022, *arXiv:2203.05482*.
- [20] M. Ashiga, W. Jie, F. Wu, V. Voskanyan, F. Dinmohammadi, P. Brookes, J. Gong, and Z. Wang, "Ensemble learning for large language models in text and code generation: A survey," 2025, *arXiv:2503.13505*.
- [21] B. Haugset and T. Stalhane, "Automated acceptance testing as an agile requirements engineering practice," in *Proc. 45th Hawaii Int. Conf. Syst. Sci.*, Jan. 2012, pp. 5289–5298, doi: [10.1109/HICSS.2012.127](https://doi.org/10.1109/HICSS.2012.127).
- [22] N. Phakdee, W. Sisomboon, and J. Arbkng, "Adopted acceptance test-driven development (ATDD) to produce robotic process automation (RPA) for reducing teaching workload," in *Proc. 28th Asia-Pacific Conf. Commun. (APCC)*, Nov. 2023, pp. 237–243, doi: [10.1109/APCC60132.2023.10460654](https://doi.org/10.1109/APCC60132.2023.10460654).
- [23] K. Andriadi, H. Soeparno, F. L. Gaol, and Y. Arifin, "The impact of shift-left testing to software quality in agile methodology: A case study," in *Proc. Int. Conf. Inf. Manage. Technol. (ICIMTech)*, Aug. 2023, pp. 259–264, doi: [10.1109/ICIMTech59029.2023.10277919](https://doi.org/10.1109/ICIMTech59029.2023.10277919).
- [24] International Software Testing Qualifications Board (ISTQB), *Certified Tester Foundation Level (CTFL) Syllabus V4.0.1*, ISTQB Association, Brussels, Belgium, Sep. 2024.
- [25] International Software Testing Qualifications Board (ISTQB), *Certified Tester Advanced Level Test Analyst (CTAL-TA) Syllabus V4.0*, ISTQB Association, Brussels, Belgium, May 2025.
- [26] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 1877–1901, 2020.
- [27] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, San Francisco, CA, USA, 2018. [Online]. Available: <https://arxiv.org/pdf/2012.11174>
- [28] G. Yenduri, M. Ramalingam, G. C. Selvi, Y. Supriya, G. Srivastava, P. K. R. Maddikunta, G. D. Raj, R. H. Jhaveri, B. Prabadevi, W. Wang, A. V. Vasilakos, and T. R. Gadekallu, "GPT (generative pre-trained transformer)—A comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions," *IEEE Access*, vol. 12, pp. 54608–54649, 2024, doi: [10.1109/ACCESS.2024.3389497](https://doi.org/10.1109/ACCESS.2024.3389497).
- [29] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.
- [30] S. Gupta, R. Ranjan, and S. Narayan Singh, "A comprehensive survey of retrieval-augmented generation (RAG): Evolution, current landscape and future directions," 2024, *arXiv:2410.12837*.
- [31] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.
- [32] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 3929–3938.
- [33] G. Izcard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. 16th Conf. Eur. Chapter Assoc. Comput. Linguistics: Main Volume*, 2021, pp. 874–880.

- [34] F. Petroni, A. Piktus, A. Fan, P. Lewis, M. Yazdani, N. De Cao, J. Thorne, Y. Jernite, V. Karpukhin, J. Maillard, V. Plachouras, T. Rocktäschel, and S. Riedel, "KILT: A benchmark for knowledge intensive language tasks," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2021, pp. 2523–2544.
- [35] S. Borgeaud et al., "Improving language models by retrieving from trillions of tokens," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 2206–2240.
- [36] K. Shuster, J. Xu, M. Komeili, D. Ju, E. Michael Smith, S. Roller, M. Ung, M. Chen, K. Arora, J. Lane, M. Behrooz, W. Ngan, S. Poff, N. Goyal, A. Szlam, Y.-L. Boureau, M. Kambadur, and J. Weston, "BlenderBot 3: A deployed conversational agent that continually learns to responsibly engage," 2022, *arXiv:2208.03188*.
- [37] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," 2023, *arXiv:2310.11511*.
- [38] A. Joseph Oche, A. Glory Folashade, T. Ghosal, and A. Biswas, "A systematic review of key retrieval-augmented generation (RAG) systems: Progress, gaps, and future directions," 2025, *arXiv:2507.18910*.
- [39] C. Arora, T. Herda, and V. Himm, "Generating test scenarios from NL requirements using retrieval-augmented LLMs: An industrial study," in *Proc. IEEE 32nd Int. Requirements Eng. Conf. (RE)*, Jun. 2024, pp. 240–251, doi: [10.1109/RE59067.2024.00031](https://doi.org/10.1109/RE59067.2024.00031).
- [40] B. Saha, U. Saha, and M. Z. Malik, "QuIM-RAG: Advancing retrieval-augmented generation with inverted question matching for enhanced QA performance," *IEEE Access*, vol. 12, pp. 185401–185410, 2024.
- [41] Y. Wang, S. Guo, and C. Wei Tan, "From code generation to software testing: AI copilot with context-based RAG," 2025, *arXiv:2504.01866*.
- [42] T. Zhang, C. Yang, Y. Su, M. Weyssow, H. Nguyen, T. Bui, H. J. Kang, Y. Li, E. L. Ouh, L. K. Shar, and D. Lo, "Benchmarking large language models for multi-language software vulnerability detection," 2025, *arXiv:2503.01449*.
- [43] G. Budakoglu and H. Emekci, "Unveiling the power of large language models: A comparative study of retrieval-augmented generation, fine-tuning, and their synergistic fusion for enhanced performance," *IEEE Access*, vol. 13, pp. 30936–30951, 2025, doi: [10.1109/ACCESS.2025.3542334](https://doi.org/10.1109/ACCESS.2025.3542334).
- [44] M. Baqar and R. Khandia, "The future of software testing: AI-powered test case generation and validation," 2024, *arXiv:2409.05808*.
- [45] S. Hudson, S. Jit, B. Caroline Hu, and M. Chechik, "A software engineering perspective on testing large language models: Research, practice, tools and benchmarks," 2024, *arXiv:2406.08216*.
- [46] Y. Shao, Y. Huang, J. Shen, L. Ma, T. Su, and C. Wan, "Are LLMs correctly integrated into software systems?" 2024, *arXiv:2407.05138*.
- [47] A. Sudjianto, A. Zhang, S. Neppalli, T. Joshi, and M. Malohlava, "Human-calibrated automated testing and validation of generative language models," 2024, *arXiv:2411.16391*.
- [48] Y. Chen, G. Dong, Y. Zhu, and Z. Dou, "Revisiting RAG ensemble: A theoretical and mechanistic analysis of multi-RAG system collaboration," 2025, *arXiv:2508.13828*.
- [49] A. Ibiloye, "Long text language ensemble models: Extension of retrieval-augmented generation (RAG)," *OSF Preprints*, 2025, doi: [10.31219/osf.io/b94ng_v1](https://doi.org/10.31219/osf.io/b94ng_v1).
- [50] R. T. Sairaj and S. R. Balasundaram, "Ensemble learning with RAG model to reduce redundant question topics in auto-generated exam questions," *Discover Comput.*, vol. 28, no. 1, p. 161, Aug. 2025, doi: [10.1007/s10791-025-09683-2](https://doi.org/10.1007/s10791-025-09683-2).
- [51] I. Saputra, "Hybrid ensemble retrieval-augmented generation for Indonesian legal consultation with keyword boosting," *J. Novel Eng. Sci. Technol.*, vol. 4, no. 2, pp. 71–85, Jul. 2025, doi: [10.56741/jnest.v4i02.1042](https://doi.org/10.56741/jnest.v4i02.1042).
- [52] M. Neelapu, "Enhancing software testing efficiency with generative AI and large language models," *Int. J. Lead. Res. Publication*, vol. 5, no. 12, pp. 1–13, Dec. 2024, doi: [10.70528/ijlr.v5.i12.1586](https://doi.org/10.70528/ijlr.v5.i12.1586).
- [53] S. Nicoletti and P. Ciancarini, "Using LLMs and essence to support software practice adoption," 2025, *arXiv:2508.16445*.
- [54] P. Sahoo, A. Kumar Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," 2024, *arXiv:2402.07927*.
- [55] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," 2021, *arXiv:2107.13586*.
- [56] S. Souvik, "Software requirements dataset," Kaggle Inc., San Francisco, CA, USA, 2019. [Online]. Available: <https://www.kaggle.com/>
- [57] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, "Evaluation of retrieval-augmented generation: A survey," 2024, *arXiv:2405.07437*.
- [58] I. Papadimitriou, I. Gialampoukidis, and S. Vrochidis, "RAG playground: A framework for systematic evaluation of retrieval strategies and prompt engineering in RAG systems," 2025, *arXiv:2412.12322*.
- [59] Z. Qi, R. Xu, Z. Guo, C. Wang, H. Zhang, and W. Xu, "Long²-RAG: Evaluating long-context & long-form retrieval-augmented generation with key point recall," 2024, *arXiv:2410.23000*.
- [60] Z.-Y. Dai, F.-Q. Wang, C. Shen, Y.-L. Ji, Z.-Y. Li, Y. Wang, and Q. Pu, "Accuracy of large language models for literature screening in thoracic surgery: Diagnostic study," *J. Med. Internet Res.*, vol. 27, Mar. 2025, Art. no. e67488, doi: [10.2196/67488](https://doi.org/10.2196/67488).
- [61] E. L. Epstein, J. Winnicki, T. Sornwanee, and R. Dwaraknath, "LLMs are overconfident: Evaluating confidence interval calibration with FermiEval," 2025, *arXiv:2510.26995*.



WANTANA SISOMBOON received the B.I.S. degree in management information systems from Walailak University, Thailand, in 2002, and the M.S. degree in management of information technology from the Prince of Songkla University, Thailand, in 2007. She is currently pursuing the Ph.D. degree with Khon Kaen University, Thailand.

She is also a Lecturer with the Faculty of Informatics, Burapha University, Thailand. Her research interests include gamification, robotic process automation, software processes, software testing, artificial intelligence, and large language models. During her Ph.D. studies, she received the Best Paper Award for her research, "AI-Driven Prompt Templates for User Acceptance Test Case Generation," from the DSAI 2024: 2nd International Conference on Data Science and Artificial Intelligence in Medan, Indonesia.



JAKKRIT KAEWYOTHA received the B.Sc. degree (Hons.) in information communication and technology and the M.S. and Ph.D. degrees in information technology from Khon Kaen University, Thailand, in 2011, 2014, and 2023, respectively. His research interests include optimization and evolutionary algorithms, with additional interests in image processing, data mining, and automated software testing.



WARARAT SONGPAN received the B.Sc. degree (Hons.) in computer science and the B.Eng. degree in computer engineering from Khon Kaen University, Thailand, in 2000 and 2004, respectively, and the Ph.D. degree in computer engineering from Chulalongkorn University, Thailand, in 2010. During her Ph.D. studies, she received a scholarship under the Development of Faculty Staff in Shortage Areas Program. She was a Visiting Researcher at NICTA in Sydney, Australia. She has contributed to projects in artificial intelligence, machine learning, software testing, educational technology, and digital transformation in higher education and industry in Thailand. Her research interests include artificial intelligence, machine learning, ensemble models, sentiment analysis, and applied intelligence systems.

...